

Fault-Tolerant Resource Analysis of Grover's Algorithm on AES Under Alternate Architectures

Emma Storberg

Computational Science: Physics
60 ECTS study points

Department of Physics
Faculty of Mathematics and Natural Sciences

Spring 2026



Emma Storberg

Fault-Tolerant Resource Analysis of
Grover's Algorithm on AES Under
Alternate Architectures

Supervisors:

Martin Strand

Joakim Bergli

Thomas Gregersen

Nadia Slavila Larsen

Abstract

Grover’s algorithm offers a quadratic speedup over classical brute-force search and is commonly cited as a quantum threat to AES. Jaques [1] argues that a physically realizable implementation capable of breaking AES-128 is essentially impossible, on the grounds of three constraints: the physical-to-logical qubit overhead of current error-correcting codes, the prohibitive cost of fault-tolerant gate implementation in these codes, and the poor parallelism of Grover’s algorithm. Since the first two pillars are explicitly anchored to the surface code, this thesis asks whether his conclusion survives under the more resource-efficient Bivariate Bicycle (BB) code family. Using the stabilizer formalism for classical simulation, we compare the BB code against the surface code at the level of quantum memory, transversal gate noise, and magic state distillation. The BB code reduces physical qubit overhead by roughly an order of magnitude and achieves lower logical failure rates throughout the practically achievable error rate regime. Despite this, Jaques’ conclusion survives: None of these improvements reduce the serial T -gate count or the time each magic state factory takes to produce a single magic state, leaving the runtime bottleneck intact. The binding constraint on a Grover-based attack on AES is not the choice of error-correcting code, but the T -gate cost of the oracle.

Acknowledgements

The inspiration for this thesis was sparked during conversations I had with Martin over the course of a summer at the Norwegian Defence Research Establishment. It was through those discussions, alongside the guidance of Thomas and Nadia, that I landed on a problem statement that allowed me to combine all the topics I was interested in and place them in a cryptographically relevant context. I am grateful to all three for their supervision throughout the project. I especially want to thank Joakim for meeting with me every week and offering his continual support. This project would not have been possible without you.

Over the course of this project, I have received invaluable help from a number of people. A particular thank you to Samuel Jaques, whose work forms the foundation of much of this thesis and who was generous enough to discuss it with me directly. Thank you also to Delphine Martres, Jonas Lidal and Tron Omland for taking the time to meet with me and answer my questions.

Finally, thank you to my good friends Jonatan, Jonas and Gaute for reading drafts and offering corrections.

— *Emma, June 2026*

Declaration of AI Usage

Artificial intelligence tools were used in a supporting role throughout this project. For code, GitHub Copilot and Claude Code were used extensively to streamline my coding workflow, especially for generating and formatting plotting code. For learning, Claude Sonnet 4.6 and ChatGPT (free tier) were used as supplementary learning aids alongside academic literature, providing intuitive explanations of concepts. In writing, the same tools were used in an editorial capacity: for word choice and synonym suggestions, for reviewing drafts to catch errors, and occasionally for proposing restructurings or rephrasings of existing text. They were also used as search aids for queries too specific or technical for conventional search engines, and to format BibTex entries for references from sources that did not provide a downloadable citation file.

I have reviewed and verified all AI-assisted content in this thesis. The arguments and conclusions presented are entirely my own, and I take full responsibility for its content.

Contents

1	Introduction	1
1.1	Thesis Structure	3
1.2	Quantum Computing Prerequisites and Notation	3
2	Background	5
2.1	The Case Against the Quantum Threat to AES	5
2.1.1	Extending the Analysis	6
2.2	Grover's Algorithm for Solving Unstructured Search.	7
2.2.1	The Algorithm and Circuit	8
2.2.2	Practical Application and Limitations of Grover's Algorithm	11
2.3	Quantum Error Correction	13
2.3.1	Toric Surface Code	14
2.3.2	Bivariate Bicycle Code: An Alternative to Toric Surface Code	17
2.4	Fault-tolerant Quantum Computation	20
2.4.1	Universal Gate Sets	20
2.4.2	Choosing a Gate Set: Clifford+T	21
3	Experiments and Analysis	25
3.1	General Limitations when Simulating Quantum Systems	25
3.1.1	Proof-of-concept Experiments with Smaller Codes	27

Contents

3.2	Utilizing Stabilizer Formalism as a Computational Tool	30
3.2.1	Simulating Stable Quantum Memory with PanQEC	31
3.2.2	Simulating Transversal Logical Operators with PanQEC	33
3.3	The Trouble with the T -gate	36
3.4	Assessing the Quantum Threat	37
4	Conclusion	43
4.1	Future Work	44
	Bibliography	46
A	Definitions of Smaller Error Correcting Codes Used	51
A.1	The 5-Qubit Code	51
A.2	The 7-Qubit Steane Code	52

Chapter 1

Introduction

In recent years, quantum algorithms have garnered significant attention for their potential to disrupt traditional cryptographic systems. The security of practically all important communication channels, including the internet as we know it, is ensured by classical cryptographic protocols based on the hardness of certain mathematical problems. With the rise of quantum technology, new algorithms have been found that exploit quantum mechanical principles, such as superposition and entanglement, to solve these problems exponentially faster than classical algorithms. Still, the implementation of quantum algorithms for cryptographic purposes continues to be largely theoretical. A quantum computer capable of breaking the most-used algorithms today is yet to be constructed, as there are many practical limitations to doing so. Thus, practical quantum computers, with thousands of qubits and low error rates, remain an area of active research; there is considerable anticipation surrounding the eventual realization of the theoretical promises associated with quantum computing. As we move into the post-quantum era, it will become increasingly important to assess our cryptographic protocols in light of the most recent developments in the field in order to protect critical infrastructure and assets.

Brute-force search is a technique that may be employed to break cryptographic systems with *symmetric* keys, which means the same keys are used for encryption and decryption [2], as opposed to *asymmetric* algorithms, which use separate encryption and decryption keys. Performing such a search involves quite simply checking every single possible key option until the right key is found, an inordinately time-consuming process if the key space is large enough. Our current protocols ensure security against these searches by having key lengths large enough that it is considered computationally infeasible for a classical computer to identify one in any reasonable amount of time. A closer look at the popular symmetric-key algorithm *Advanced Encryption Standard* (AES) [3], which encrypts data by applying a secret key through a series of mathematical transformations, exemplifies this: It exists in versions that use key sizes of 128, 192 and 256 bits, resulting in search spaces of 2^{128} , 2^{192} and 2^{256} possible keys respectively.

A relevant quantum algorithm here is *Grover's algorithm* [4], in which a procedure is given for a quantum computer to perform a search much faster than brute-force search can be done on classical computers, thereby potentially threatening the security of AES. As we will see, Grover's algorithm offers a quadratic improvement over classical algorithms, meaning the number of operations needed when we use this quantum algorithm is on the order of the square-root of the number of operations required when solving the same problem classically. However, finding the exact runtime is not as straightforward as the asymptotic estimates, because there are constant multiplicative factors hidden within the asymptotic notation that in practice determine the true resource cost of each operation. Asymptotic estimates also do not take into account what kind of parallelization may be possible. Analyzing the true cost of using Grover's algorithm is therefore a nontrivial problem that warrants further exploration, which constitutes the core of this project.

Building a quantum machine in practice is no easy feat. The root of the issue is that quantum information is extremely fragile. The qubits that store information are unstable and very vulnerable to noise that changes the value the qubit holds. We therefore wish to construct circuits of multiple qubits that work together to protect the information from errors, such that they form a single logical qubit that can have its errors located and corrected, making them more stable. This approach, though useful, will often require large numbers of qubits, which poses an engineering challenge to keep them all as stable as possible, facilitate connections between them, and perform computations on the logical qubits rather than on the physical qubits individually.

Jaques considers this in his 2024 invited talk about Grover's algorithm as a quantum attack on AES [1], in which he argues that due to the error correction models and gate sets currently in widespread use, a physical implementation of Grover's algorithm will never see the light of day on anything resembling current quantum architectures. His argument rests on three pillars: the enormous physical-to-logical qubit ratio imposed by current error-correcting codes, the prohibitive cost of fault-tolerant gate implementation under those codes, and the poor parallelism of Grover's algorithm itself. Since the first two pillars are explicitly tied to the surface code, the dominant error-correcting architecture in current fault-tolerant designs, a natural question arises: Does his conclusion survive under a more favorable architecture? This thesis investigates that question by examining what Jaques' analysis looks like under the assumption of the Bivariate Bicycle (BB) code family [5], a recent class of quantum low-density parity-check (LDPC) codes that offer substantially better qubit efficiency than the surface code.

A central challenge in this investigation is that quantum systems are notoriously difficult to simulate classically. If we could simulate quantum systems efficiently in general, it would beg the question why we need to realize them at all. This limits how directly we can study fault-tolerant implementations of Grover's algorithm: Even the smallest nontrivial instances quickly exceed the memory available on consumer machines. This

thesis navigates that barrier using the stabilizer formalism [6, 7] as a computational tool, which enables efficient classical simulation of the error-correcting codes and logical gate operations relevant to our analysis, without requiring full statevector simulation.

The specific contributions of this thesis are as follows: We verify that fault-tolerant logical computation in the Steane code works as intended by implementing a complete instance of Grover’s algorithm on two error-corrected logical qubits under depolarizing noise. We then compare the BB code against the surface code at the level of quantum memory and transversal gate noise using the PanQEC simulation framework, and interpret the results of Xu et al. [8] on BB code magic state distillation factories in the context of Jaques’ resource estimates. Together, these results allow us to assess which of Jaques’ three pillars are weakened by adopting the BB code, and whether any are broken.

1.1 Thesis Structure

The remainder of this thesis is organized as follows: Chapter 2 develops the theoretical background, covering Grover’s algorithm, quantum error correction, the surface and BB codes, and logical gates in these codes. Chapter 3 describes the experiments: a fault-tolerant implementation of Grover’s algorithm on the 7-qubit code as a proof-of-concept, followed by PanQEC-based comparisons of the BB code and surface code at the level of quantum memory and transversal gate noise, and magic state distillation analysis drawing on Xu et al. [8]. Chapter 4 synthesizes the findings and returns to Jaques’ three pillars to assess which survive under the BB code architecture. All code experiments and simulations can be found on GitHub¹.

1.2 Quantum Computing Prerequisites and Notation

We write $\mathbb{I}_m$ for the $m \times m$ identity matrix. The set of complex $m \times n$ matrices is denoted $\mathbb{C}^{m \times n}$, and $\mathbb{R}$ and $\mathbb{C}$ denote the real and complex numbers respectively.

Throughout this text, n denotes the number of qubits in a system, and we write $N = 2^n$ for the dimension of the corresponding Hilbert space. We assume familiarity with Dirac notation: A quantum state is written as a ket $|\psi\rangle \in \mathbb{C}^N$, its dual as a bra $\langle\psi|$, and the inner product of two states as $\langle\phi||\psi\rangle$. Dirac notation is occasionally used outside of its quantum mechanical context as a convenient algebraic shorthand: $|i\rangle\langle j|$ denotes the outer product of two basis vectors, defining an $N \times N$ matrix with a 1 in position (i, j) and zeroes elsewhere, with zero-based indices throughout.

¹<https://github.com/emmastorberg/CS5960PHYS.git>

A single qubit occupies a two-dimensional Hilbert space spanned by the computational basis states $|0\rangle$ and $|1\rangle$. A general single-qubit state is a superposition

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1.$$

More generally, for an n -qubit system, the 2^n computational basis states $|x\rangle$, $x \in \{0, 1\}^n$, span the full Hilbert space, and the state of the system is a superposition $|\psi\rangle = \sum_x c_x |x\rangle$ with $c_x \in \mathbb{C}$. The probability of obtaining outcome x upon measurement in the computational basis is $|c_x|^2$, so the normalization condition $\sum_x |c_x|^2 = 1$ ensures that the probabilities sum to 1.

Quantum gates are represented by unitary matrices acting on these state vectors. The four single-qubit gates used most frequently in this text are the Pauli operators:

$$\mathbb{I}_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

These are sometimes written as $\sigma_0, \sigma_1, \sigma_2, \sigma_3$ (or $\sigma_x, \sigma_y, \sigma_z$ for the non-identity operators) in the physics literature. Where this text departs from the $\mathbb{I}_2, X, Y, Z$ convention, in particular in the discussion of stabilizer codes in Section 2.3.1, the notation is introduced locally.

Another gate of central importance is the single-qubit Hadamard gate:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

which maps each computational basis state to an equal superposition: $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. Applying H to every qubit of an n -qubit register initialized in $|0\rangle^{\otimes n}$ (the n -fold tensor product $|0\rangle^{\otimes n} = |0\rangle \otimes \cdots \otimes |0\rangle$) produces the uniform superposition $\frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$, a construction that appears repeatedly in quantum algorithms, including Grover's algorithm.

Chapter 2

Background

2.1 The Case Against the Quantum Threat to AES

All information in this section is drawn from Jaques [1].

A superficially alarming picture of AES's vulnerability to quantum computers is easy to construct. Asymptotically, Grover's algorithm offers a square-root speedup over classical brute-force search: Where a classical computer must check on the order of 2^{128} keys to break AES-128, Grover's requires only half as many quantum operations, on the order of 2^{64} . This purported cost reduction of an attack on AES by way of Grover's algorithm gives the impression that the scheme is under threat, with its security effectively halved.

Early estimates of the quantum circuit implementing AES as part of Grover's algorithm suggested it would require only a few thousand qubits, which sounds comparable to hardware specifications already being announced. The natural conclusion seems to be that doubling key sizes to AES-256 is an urgent priority, or that AES-128 is already under serious threat. Jaques systematically dismantles this conclusion. He identifies three distinct misconceptions, each of which individually complicates the picture, and whose combination makes a successful Grover-based attack on AES-128 appear essentially physically impossible.

The first misconception is that qubit count is the binding constraint that prevents the construction of cryptographically relevant quantum computers. The early AES circuit estimates cited a figure of roughly 2,953 qubits, with an important caveat that this refers to the logical qubit count, not physical devices. Logical qubits are an abstraction encoded across thousands of noisy physical qubits using error-correcting codes, which is necessary because quantum hardware is corrupted by any uncontrolled interaction with its environment. Under the surface code, the dominant architecture in current fault-tolerant designs, the ratio between physical and logical qubits is not a small overhead, meaning

an attack requiring thousands of logical qubits could demand millions of physical ones. Thus, we are further away from the required physical resources than it may seem initially.

The second misconception is that the square-root speedup translates cleanly into a recommendation to double key sizes. Any realistic classical attack and its associated estimated runtimes contain the hidden assumption that we run the algorithm in parallel. In the classical case, parallelism is essentially free; we can divide the key space among P machines and the time drops by a factor of P at no extra cost. However, when we assume Grover's has a square-root speedup over the classical runtime, we cannot simply assume we have access to the same kind of parallelism. In fact, as we will see, the total number of operations across all machines to run Grover's algorithm actually grows with P . Thus, once the overhead of parallelism is accounted for, Jaques argues that the effective cost of breaking AES-128 rises above the naive 2^{64} figure.

The third misconception is that the constant factor in the quantum attack's cost is small and well-understood. Implementing AES as a quantum circuit requires translating it into a gate set available on error-corrected hardware, and certain gates are enormously expensive in the surface code. Once the overhead of these gates and error correction cycles is included, the numbers become staggering: Even under the most permissive estimates of quantum computing capability¹ put forward by the National Institute of Standards and Technology (NIST), the required machine would need enough qubits to cover the surface of the moon, at a minimum energy cost equivalent to a full year's output of a nuclear power plant. Jaques' conclusion is simply that this will never be built.

The broader point is not that AES is theoretically unbreakable by quantum algorithms. Rather, Jaques addresses three hurdles to real-world implementation (qubit overhead in logical circuits, poor parallelism and high gate implementation costs) that together indicate that the threat posed by Grover's algorithm to AES-128, as commonly framed, is not a serious near-term concern. The intuitive appeal of square root speedup resulting in half the security collapses once parallelism, error correction, and circuit implementation costs are considered in full rather than assumed away.

2.1.1 Extending the Analysis

The second pillar of Jaques' argument, regarding the poor parallelism of Grover's, rests on a property of the algorithm itself and is independent of architectural assumptions. The first and third pillars, however, are both necessarily tied to the current state of hardware: The physical-to-logical qubit ratio depends on the choice of error-correcting code, and the gate implementation cost is explicitly anchored to the surface code. Jaques acknowledges this himself, noting that future architectures may shift the relevant cost

¹atomic-scale qubits with speed-of-light propagation running for a millennium

metrics in ways that are difficult to predict today. The following sections introduce the background necessary to extend his analysis under the assumption of alternative error correcting codes, namely the BB code family [5].

2.2 Grover's Algorithm for Solving Unstructured Search

Section 2.2 is adapted from parts of my previous work [9], with revisions and additions for clarity.

Formally, the computational problem that Grover's algorithm addresses is a quite general problem known as *unstructured search* [10]. It can be described as follows:

The unstructured search problem

Let Σ denote the binary alphabet $\{0,1\}$. Suppose we are given a function $f : \Sigma^n \rightarrow \Sigma$ that we can compute efficiently. We wish to determine the binary strings $x \in \Sigma^n$ such that $f(x) = 1$, or report that no solution exists.

We call this search problem *unstructured* because f can be an arbitrary function, meaning there is no underlying pattern or constraint, often called a *promise* [10], that we can take advantage of to systematically search for a solution string x , if one even exists. This means that in the classical approach to solving this problem, we can be sure of the solution only after checking each of the $2^n = N$ possible strings, thus requiring N queries.

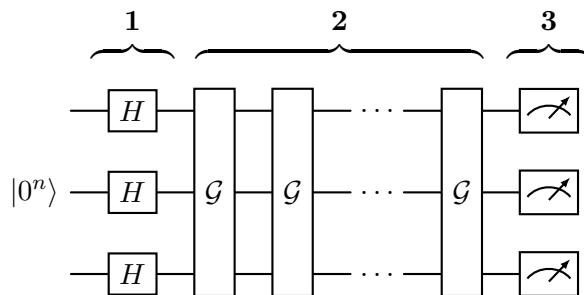


Figure 2.1: Circuit diagram of Grover's algorithm, which essentially consists of three main parts: initialization of n qubits into an equal superposition (1), followed by t Grover iterations (2), and finally measurement to reveal a candidate solution (3).

2.2.1 The Algorithm and Circuit

The circuit for Grover's algorithm is shown in Fig. 2.1. We begin by applying H to each of the n input qubits, producing the uniform superposition $\frac{1}{\sqrt{N}} \sum_{x \in \Sigma^n} |x\rangle$ over all N keys simultaneously. Next, the sequential application of the Grover operation $\mathcal{G}$ selectively amplifies the amplitude of the correct key before we measure to obtain a solution.

The Grover operation $\mathcal{G}$

The gate sequence needed for the Grover operation $\mathcal{G}$ is:

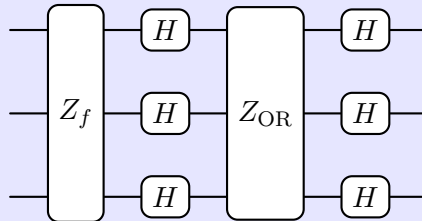
$$H^{\otimes n} Z_{\text{OR}} H^{\otimes n} Z_f$$

where Z_f is the phase query gate for the function f , and Z_{OR} is the phase query gate for the n -bit logical OR function. They are defined as follows:

$$Z_f : |x\rangle \mapsto (-1)^{f(x)} |x\rangle \quad \forall x \in \Sigma^n$$

$$Z_{\text{OR}} : \begin{cases} |x\rangle & x = 0^n \\ -|x\rangle & x \neq 0^n \end{cases}$$

Since the gates are applied from right to left, we apply Z_f first, followed by H on all qubits, then Z_{OR} , and finally another set of Hadamard gates. This gate sequence has two conceptually distinct parts, which are worth naming explicitly. The first is the *oracle* Z_f , which is a quantum circuit that evaluates the function f and encodes the result as a phase, marking solution states with a sign flip without measuring or disturbing them. The second is the *diffusion operator*, implemented by $H^{\otimes n} Z_{\text{OR}} H^{\otimes n}$. It increases the amplitude of marked states relative to unmarked ones by reflecting the state vector about the uniform superposition. Together, the oracle and diffusion operator constitute a single Grover operation. The circuit diagram of the full operation $\mathcal{G}$ on three qubits looks like this:



Note that only the oracle requires a query to f . Thus, the number of queries to f we need corresponds to the number of iterations t of $\mathcal{G}$ we perform.

2.2. Grover's Algorithm for Solving Unstructured Search

A natural way to use the algorithm for search problems is to choose a number of iterations t of the Grover operation, run the circuit to obtain a candidate solution x , and check if x is a solution (in this case whether $f(x) = 1$). If so, we return x , otherwise we run the algorithm again (possibly with a different value for t) or return “no solution”. This sounds simple enough, but still, important questions remain unanswered: What is the “suitable” number of operations t and how do we identify it?

To understand what is happening here, it is helpful to separate the possible solution strings in Σ^n into two categories, which we will call solutions and nonsolutions. These sets can be described as follows:

$$\begin{aligned} A_0 &= \{x \in \Sigma^n : f(x) = 0\} \\ A_1 &= \{x \in \Sigma^n : f(x) = 1\} \end{aligned}$$

where A_0 contains all the strings that evaluate to 0 when given as input to f , and A_1 has the strings we seek that evaluate to 1. In particular, we are interested in uniform superpositions over these two sets, defined as:

$$\begin{aligned} |A_0\rangle &= \frac{1}{\sqrt{|A_0|}} \sum_{x \in A_0} |x\rangle \\ |A_1\rangle &= \frac{1}{\sqrt{|A_1|}} \sum_{x \in A_1} |x\rangle \end{aligned}$$

Importantly, we will assume from here on that A_0 and A_1 are both nonempty. However, we can still easily see what happens in the last two cases (all strings evaluate to 0, or all strings are solutions), as the former will return no solution, and in the latter case, any string is a solution. Since we are able to handle these special cases, we can safely continue our analysis under this assumption.

A central observation here is the fact that we can write the initialized uniform superposition as a linear combination of the states $|A_0\rangle$ and $|A_1\rangle$:

$$H^{\otimes n} |0^n\rangle = \sqrt{\frac{|A_0|}{N}} |A_0\rangle + \sqrt{\frac{|A_1|}{N}} |A_1\rangle$$

The key idea we utilize is this: **The state of the qubit register remains in the subspace spanned by $|A_0\rangle$ and $|A_1\rangle$ after every application of $\mathcal{G}$.** Thus, we would like to process the initialized state $H^{\otimes n} |0^n\rangle$ (with uniform amplitude across all N components) such that through successive iterations, we nudge the amplitude of the solution component $|A_1\rangle$ slightly higher, while lowering the amplitude of the nonsolutions $|A_0\rangle$. Provided we find the suitable number of iterations t , we can push the amplitude of the state corresponding to the solutions arbitrarily high, and consequently see a high probability that this state is measured.

We can show that by applying $\mathcal{G}$ to $|A_0\rangle$ and $|A_1\rangle$, we obtain the following linear combinations [10]:

$$\begin{aligned}\mathcal{G}|A_0\rangle &= \frac{|A_0| - |A_1|}{N}|A_0\rangle + \frac{2\sqrt{|A_0| \cdot |A_1|}}{N}|A_1\rangle \\ \mathcal{G}|A_1\rangle &= -\frac{2\sqrt{|A_0| \cdot |A_1|}}{N}|A_0\rangle + \frac{|A_0| - |A_1|}{N}|A_1\rangle\end{aligned}$$

This clearly demonstrates that we are still in the span of $|A_0\rangle$ and $|A_1\rangle$ after an application of $\mathcal{G}$. Moreover, we can describe the action of $\mathcal{G}$ as a rotation matrix, and we can rewrite it as the square of a somewhat simpler-looking matrix [10]:

$$\begin{bmatrix} \frac{|A_0| - |A_1|}{N} & -\frac{2\sqrt{|A_0| \cdot |A_1|}}{N} \\ \frac{2\sqrt{|A_0| \cdot |A_1|}}{N} & \frac{|A_0| - |A_1|}{N} \end{bmatrix} = \begin{bmatrix} \sqrt{\frac{|A_0|}{N}} & -\sqrt{\frac{|A_1|}{N}} \\ \sqrt{\frac{|A_1|}{N}} & \sqrt{\frac{|A_0|}{N}} \end{bmatrix}^2$$

Now it is easier to see the similarity to the general form of a rotation matrix, with $\theta = \sin^{-1}\left(\sqrt{\frac{|A_1|}{N}}\right)$:

$$\begin{bmatrix} \sqrt{\frac{|A_0|}{N}} & -\sqrt{\frac{|A_1|}{N}} \\ \sqrt{\frac{|A_1|}{N}} & \sqrt{\frac{|A_0|}{N}} \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Since squaring a rotation is the same thing as applying it twice, we return to the original rotation matrix for $\mathcal{G}$, and write it as:

$$\begin{bmatrix} \frac{|A_0| - |A_1|}{N} & -\frac{2\sqrt{|A_0| \cdot |A_1|}}{N} \\ \frac{2\sqrt{|A_0| \cdot |A_1|}}{N} & \frac{|A_0| - |A_1|}{N} \end{bmatrix} = \begin{bmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{bmatrix}$$

with θ defined as before. Thus, we have found a rotation matrix that describes the action of $\mathcal{G}$ on the subspace spanned by A_0 and A_1 . This also means that the state of the qubit register after initialization can be described as:

$$H^{\otimes n}|0^n\rangle = \sqrt{\frac{|A_0|}{N}}|A_0\rangle + \sqrt{\frac{|A_1|}{N}}|A_1\rangle = \cos \theta |A_0\rangle + \sin \theta |A_1\rangle$$

So, each time we apply $\mathcal{G}$, the state of the register is rotated by 2θ , and we can find a closed-form expression for the state after t applications of $\mathcal{G}$. For brevity, we will henceforth refer to the initialized qubit register $H^{\otimes n}|0^n\rangle$ as $|u\rangle$.

$$\mathcal{G}^t|u\rangle = \cos((2t+1)\theta)|A_0\rangle + \sin((2t+1)\theta)|A_1\rangle$$

This expression makes sense intuitively: 2θ for each application of $\mathcal{G}$, and one additional θ to account for the state we started in. Now recall that for any quantum state of the form $\alpha|A_0\rangle + \beta|A_1\rangle$, we measure a solution $x \in A_1$ with probability $|\beta|^2$. With the state in question $\mathcal{G}^t|u\rangle = \cos((2t+1)\theta)|A_0\rangle + \sin((2t+1)\theta)|A_1\rangle$, the probability of measuring a solution after t iterations is $\sin^2((2t+1)\theta)$. This is the probability we want to maximize

in order to find a solution to the search problem with Grover's algorithm, and as such, we call $|A_1\rangle$ our *target state* [10].

Another consideration is that we would also like to minimize t , as this is the number of queries to f we have to make. Thus, to reach our target state with a probability close to 1 while minimizing t , we aim to find t such that $(2t + 1)\theta \approx \frac{\pi}{2}$, since this is the smallest angle where the square of the sine function equals 1. In terms of t , we aim for $t \approx \frac{\pi}{4\theta} - \frac{1}{2}$. We cannot guarantee that this value of t is an integer, so we round down to find the number of iterations t , giving $t = \lfloor \frac{\pi}{4\theta} \rfloor$. For the algorithm to work properly, it is also important that we do not iterate past the recommended iteration number, lest we “overshoot” our solution by rotating past it. If we simply continue applying $\mathcal{G}$, we will in fact decrease the probability that the solution state is measured, and over time, we will oscillate between solutions and nonsolutions as we rotate the register state around the unit circle [10].

Up to this point, we have studied a general version of Grover's algorithm that can be applied to a number of search problems, ranging from those we know to have a single solution, a larger solution set of a fixed size, or even an unknown number of solutions. We have determined that an iteration number of $t = \lfloor \frac{\pi}{4\theta} \rfloor$ (and no more) will maximize the probability of finding one of these solutions, but what we must also consider is the fact that this way of setting iteration number t also depends on how many solutions there are, as θ depends on the size of the set A_1 . Fortunately, since we are considering only the cryptographic application of using the algorithm to search for a single key from some symmetric cryptographic protocol, we can restrict ourselves to the cases where we know there is exactly one solution (or one cryptographic key).

Unique search, as this problem is formally known, is equivalent to the general unstructured search problem, with the additional promise that there is exactly one solution string x . Recall that in the general case, $\theta = \sin^{-1} \left(\sqrt{\frac{|A_1|}{N}} \right)$. Now that we have determined there is only a single solution, we know that $|A_1| = 1$. Thus, for large N , we see that $\theta = \sin^{-1} \left(\sqrt{\frac{1}{N}} \right) \approx \sqrt{\frac{1}{N}}$, since for smaller angles, the sine function starts to look more like the identity function. If we use this approximation of θ in our previously determined expression $t = \lfloor \frac{\pi}{4\theta} \rfloor$, we get $t = \lfloor \frac{\pi}{4} \sqrt{N} \rfloor$. Since t represents queries to the function f , we see the quadratic improvement mentioned earlier: The query complexity decreases from $\mathcal{O}(N)$ classically to $\mathcal{O}(\sqrt{N})$ with Grover's algorithm.

2.2.2 Practical Application and Limitations of Grover's Algorithm

At first glance, the improvements afforded by Grover's algorithm seem substantial. Though not a perfect algorithm in the sense that there is a probabilistic element and

some chance of error², Grover’s algorithm clearly has an advantage in runtime complexity over the classical strategy. Therefore, we identify that there is some trade-off between computational efficiency and certainty of outcome when choosing to use Grover’s to tackle the unstructured search problem. In terms of correctness, it can be proven analytically that probability of success when using Grover’s is always greater than or equal to $1 - \frac{1}{N}$ [10], meaning success is typically likely, particularly as N grows large. We are also free to run the algorithm in its entirety as many times as we choose, and if any one of them succeeds, it will identify a solution, with a high chance of doing so using fewer queries to f than its classical counterpart. With the classical cost on the order of 2^{128} for AES-128, a naive calculation finds the new cost with Grover’s square-root speedup to be around 2^{64} , perhaps with some small constant to account for overhead and setting up the quantum circuit. However, this estimation does not take into account a number of practical concerns and considerations.

First, Grover’s algorithm, as described in Section 2.2.1, uses an oracle f , which in our case corresponds to the encryption algorithm in AES. A complicated scheme in its own right, AES must be implemented in the quantum hardware using computationally expensive gates and structures [1, 11], introduced and discussed in more detail in Section 2.4.1. This adds significant cost to the runtime of the algorithm [1], which the commonly cited Big O abstraction does not capture. Exploring circuit designs of AES and their costs is outside of the scope of this project, but it is an active field of research [12, 13, 11] that will be crucial for full asymptotic analysis and practical implementation of Grover’s algorithm applied to AES.

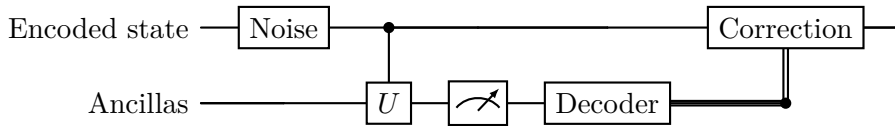
Secondly, as pointed out by Jaques, any realistic attack will require parallelization [1], and unfortunately, Grover’s algorithm does not parallelize well. If we consider the full key space of N keys and partition it into P subsets, we can assign each of the P partitions to a machine responsible for a search space containing $\frac{N}{P}$ keys. With the square-root speedup over the classical method as discussed, we can now find the key in $\mathcal{O}(\sqrt{\frac{N}{P}})$ time. When comparing this to serial Grover’s with a runtime of $\mathcal{O}(\sqrt{N})$, we find that parallelizing only reduces the runtime by a factor of $\sqrt{P}$, not P . Worse yet, the additional partitions add an overhead, with the total cost of operations going up to $P \cdot \mathcal{O}(\sqrt{\frac{N}{P}}) = \mathcal{O}(\sqrt{P \cdot N})$. Thus, in our case with AES-128, the bad parallelism of Grover means its cost is certainly higher than the naive estimation of 2^{64} .

The parallelization argument holds for any algorithm on the order of $\sqrt{N}$, and it turns out $\mathcal{O}(\sqrt{N})$ is the best we can do: For the general unstructured search problem, it has been proven that Grover’s algorithm is *asymptotically optimal* [14, 15], meaning that as the search space grows, it is not possible to find an algorithm that performs better than on the order of $\sqrt{N}$ in the worst case.

²in contrast to the classical runtime we discussed, where the exhaustive search strategy leaves no chance of error whatsoever

2.3 Quantum Error Correction

To run Grover’s algorithm in practice, or any other quantum algorithm, it is not enough to simply build the circuit in Fig. 2.1. Real quantum hardware is inherently noisy: Every gate introduces small errors, and qubits continuously interact with their environment through a process known as *decoherence* [16], meaning even when qubits are idle and simply acting as memory, the data they store can be lost or corrupted. At the scale required to evaluate the AES oracle inside Grover’s algorithm, these errors accumulate to the point where the output of the circuit becomes unreliable [1]. The prevailing approach to managing this issue is encoding logical information redundantly across many physical qubits, rather than attempting to perfect individual qubit quality to the point where errors become negligible, as the former is generally considered the easier path forward from an engineering standpoint [1]. We call this framework *quantum error correction* (QEC), the general structure of which is shown in the following circuit diagram:



In the circuit, we see that noise acts on the encoded state, and we make use of extra qubits called *ancillas*. By some process U , typically an *error-correcting code*, which we return to in Section 2.3.1, we are able to detect the errors that have occurred and store that information in the ancillary qubits, which can be measured. The result of the ancilla measurement tells us what corrections to apply to restore the logical state, all without ever directly measuring (and thus disturbing) the encoded logical state.

Unlike in a classical computer where the only errors we consider are bit-flip errors (where 0 is interpreted as 1, or vice versa), quantum information can in principle be affected by a continuous space of errors. This turns out to be more manageable than it seems at first glance. To understand why, note that the set of Pauli matrices $\{\mathbb{I}_2, X, Y, Z\}$ forms a basis for $\mathbb{C}^{2 \times 2}$ [4], meaning any error E on a single qubit can be decomposed linearly as:

$$E = \alpha \mathbb{I}_2 + \beta X + \gamma Z + \delta Y \quad \alpha, \beta, \gamma, \delta \in \mathbb{C}.$$

Although physical noise can in principle be any continuous perturbation, any such error is projected onto one of these discrete Pauli components the moment a measurement is performed. Just as measuring a superposition yields a single definite outcome, measuring a system affected by a continuous error forces it into a definite X -, Z -, or Y -error. Since $X^2 = Y^2 = Z^2 = I$, errors are corrected simply by applying the corresponding Pauli operator to the affected qubit, as identified by the syndrome measurement.

This discretization of a continuous error space into Pauli errors is powerful on its own, but we can reduce the possible error types even more by considering the relation $Y = iXZ$. A Y -error can be expressed as a simultaneous X - and Z -error, which can be corrected with independent X - and Z -corrections. Ultimately, these are the only two types of errors we must correct to span all possible single-qubit errors (up to a global phase), with X - and Z -errors corresponding to bit- and phase-flips respectively. Still, having two types of fundamental errors to correct, compared to only one classically, in combination with quantum hardware being inherently more difficult to work with from an engineering perspective, has the unfortunate result of massively complicating the circuitry required for computation as soon as any practically viable error correction measures are introduced.

The central theoretical guarantee is the *threshold theorem* [4]: If the physical error rate per gate is below a code-dependent threshold value, the logical error rate can be made arbitrarily small by increasing the number of physical qubits used to encode each logical qubit. This sounds promising, because it means that in principle, noise is not a serious problem for quantum computation. However, this comes at a steep cost. More physical qubits per logical qubit means larger, deeper circuits and higher gate counts. Exploring the challenges and tradeoffs associated with encoding information in this way is the focal point of this project, which will be revisited in more detail in Section 2.4.1.

2.3.1 Toric Surface Code

The dominant method of error correction in fault-tolerant quantum computing, and the one used by leading implementations such as Google's, is the *surface code* [17]. There is some inconsistency in the literature about what constitutes a *toric code*, a *surface code*, and a *toric surface code*. We adhere to the nomenclature set by IBM [18], where toric codes exist in a grid across the surface of a torus with periodic boundary conditions, and surface codes and toric surface codes are used interchangeably to refer specifically to the portion of a torus grid laid flat with non-periodic boundary conditions.

Following the discovery of the toric code in 1997 [19], toric surface code has remained a prevalent error-correcting architecture due to its conceptual simplicity and geometric locality, requiring only nearest-neighbor qubit interactions, which are significantly easier to engineer than long-range connections. In essence, surface code works by creating a grid of data qubits and measurement qubits interspersed and defining *stabilizers* that flag X -errors and Z -errors when they occur in specific locations. The error correcting capability lies in measuring the stabilizers across the grid repeatedly, decoding where errors have occurred, and applying the appropriate correction based on the stabilizer measurement. The stabilizer formalism can in many ways be viewed as the quantum generalization of parity checks [6, 7], which are often used similarly in classical error correction to identify where and how to restore corrupted data.

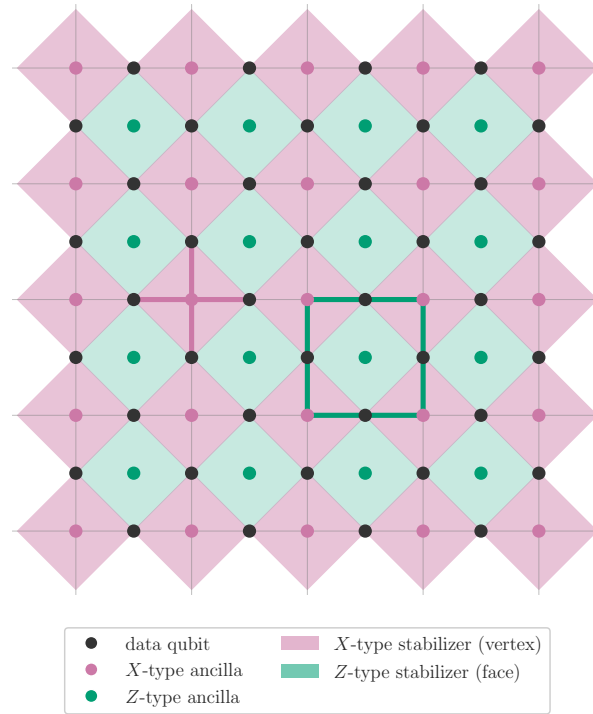


Figure 2.2: A 5×5 surface code lattice. Data qubits (black circles) lie on the edges of the grid. Each vertex v defines $\text{star}(v)$, the set of edges incident to it, and hosts a pink X -type stabilizer acting on those data qubits. Each plaquette p defines $\text{boundary}(p)$, the set of edges surrounding it, and hosts a teal Z -type stabilizer acting on those data qubits. Colored circles at every v and p are ancilla qubits used to measure the respective stabilizers. Together the stabilizers tile the lattice in a checkerboard pattern.

As an illustrative example, Fig. 2.2 shows a small surface code with a grid of these stabilizers. Data qubits lie on the edges of the grid, shown as black circles. X -type stabilizers (pink) are centered at grid vertices, hence the name *vertex stabilizers*. Each vertex v defines a set $\text{star}(v)$ consisting of the (up to four) edges incident to v , and the corresponding stabilizer acts on exactly those data qubits. One such stabilizer is highlighted with pink arms tracing $\text{star}(v)$, marking the data qubits v stabilizes. The Z -type stabilizers (green) are the square faces of the grid, each acting on the four data qubits along its boundary. Each such face p , which we call a *plaquette*, defines a set $\text{boundary}(p)$ consisting of the four edges surrounding it, and the corresponding stabilizer acts on those data qubits. One plaquette is highlighted in teal, tracing the data qubits in $\text{boundary}(p)$. The Z -type stabilizers associated with these plaquettes are known interchangeably as *face stabilizers* and *plaquette stabilizers*.

Ancillary check qubits (colored circles) sit at each vertex v and plaquette center p , measuring the X - and Z -type stabilizers respectively. Being able to use check qubits for measurements in the stabilizers are a crucial element of quantum error correction specifically, as measuring the actual state of data qubits to identify data corruption would destroy their states, and thus the data itself.

The stabilizer formalism allows us to define algebraically the units described visually in Fig. 2.2 with two sets of commuting stabilizer generators. For each vertex v in the grid, we have an X -stabilizer

$$S_v^X = \prod_{j \in \text{star}(v)} X_j,$$

which multiplies the Pauli X operator on the four qubits touching v , or fewer at the grid boundaries. We similarly define a Z stabilizer for each plaquette p as

$$S_p^Z = \prod_{j \in \text{boundary}(p)} Z_j,$$

which multiplies the Pauli Z operator on the qubits surrounding p . In general, a stabilizer code encoding k logical qubits into n physical qubits requires exactly $n - k$ independent generators, which becomes $n - 1$ in the surface code with $k = 1$. In the 5×5 example in Fig. 2.2, there are 40 data qubits and 41 defined stabilizers (25 vertex, 16 face), of which exactly 39 are independent due to one linear dependency in each type.

The groups formed by all S^X and S^Z respectively have elements that are all tensor products of Pauli operators and that commute. Together, each of these groups has a shared eigenspace constrained by

$$S_i |\psi\rangle = + |\psi\rangle \quad \forall i,$$

where S_i represents a general stabilizer (either X or Z) applied to an appropriate vertex or plaquette, denoted by i . This eigenspace contains the legal code words, or in this case, the states corresponding to logical 0 and logical 1 (henceforth denoted by $|0\rangle_L$ and $|1\rangle_L$ respectively).

The Pauli operators satisfy the anticommutation relation $\sigma_j \sigma_k + \sigma_k \sigma_j = 2\delta_{jk} \mathbb{I}_2$, where σ_i are the Pauli operators according to the naming convention $\sigma_1 = X, \sigma_2 = Y, \sigma_3 = Z$, and δ_{jk} is the Kronecker delta [20]. Thus, if an error operator E commutes with a stabilizer S_i , the eigenvalue of the surrounding qubits' state is unchanged:

$$S_i(E |\psi\rangle) = E(S_i |\psi\rangle) = +E |\psi\rangle.$$

However, if E and S_i anticommute, we will have $S_i(E |\psi\rangle) = -E(S_i |\psi\rangle)$ instead, which changes the eigenvalue of the local stabilizer state from $+1$ to -1 . Since any two Pauli operators either commute or anticommute, and since errors are always projected onto Pauli components upon measurement (as established in Section 2.3), these two cases are exhaustive.

At this point, we can do a measurement of the ancillary check qubits of the stabilizers to determine which ones violate our constraints and flag errors in their neighbors. Thinking back to the notion of parity checks, we consider the result of each stabilizer measurement

a remapping of the parity check values 0 and 1 to $+1$ and -1 respectively. This is a very convenient mapping for us, because when we work with Z operators, ± 1 happen to be the eigenvalues of the eigenstates $|0\rangle$ and $|1\rangle$. If we measure the check qubits in the stabilizers and see $|0\rangle$, we know the data qubits are in the eigenstate $+1$, and we conclude no error has occurred among the neighboring qubits of this stabilizer. However, if we see $|1\rangle$, we know the surrounding data qubits are in the eigenstate -1 , and an error has occurred here. A measurement of an X -stabilizer is slightly more involved, but still unproblematic; the relation $HXH = Z$ allows us to simply apply H to the ancillary check qubit and do a measurement as if it were a Z -stabilizer instead.

Together, the eigenvalues s_i that we observe at each of the m stabilizer measurements form a string $(s_1, s_2, \dots, s_{m-1}, s_m) \in \{\pm 1\}^m$ that we call the *syndrome* of the full state. It tells us about what errors are likely to have occurred and where, which we can use to determine what corrections we would like to apply to restore the logical state. It should be noted, however, that the syndrome cannot distinguish between no error and an even number of errors on the same stabilizer's neighbors, as both produce a $+1$ eigenvalue, leaving an undetectable error. This is a fundamental limitation of stabilizer codes generally, not specific to the surface code.

While conceptually simple, the toric surface code has a clear drawback in that it is very qubit-hungry, with estimates predicting on the order of 1000 physical qubits per logical qubit at realistic error rates [1, 21]. This has sparked further study and innovation in order to uncover more favorable schemes in terms of resource efficiency [5].

2.3.2 Bivariate Bicycle Code: An Alternative to Toric Surface Code

The continued usage of the surface code at current correction rates is one of the underlying assumptions in Jaques' predictions. Thus, it is interesting for the sake of our analysis to see how the prognosis for Grover's algorithm changes with a different choice of error correcting code. A few promising examples can be found in the *Low Density Parity Check* (LDPC) family of codes, of which the surface code is a special case. LDPC codes are stabilizer codes where each stabilizer acts on a fixed, small number of qubits, and each qubit participates in a fixed, small number of checks, both independent of the code size. For instance, in the surface code, each stabilizer acts on exactly four qubits (or fewer at boundaries) and each qubit participates in exactly two X -type and two Z -type checks. This property makes LDPC codes a strong contender for practical architectures, since stabilizer measurements remain manageable even as the code scales.

Bivariate Bicycle codes, also known as BB codes, are one of these potential candidates to replace surface codes, also in the LDPC family [5]. Though similar to surface codes at a high level of abstraction, BB codes differ in that their stabilizers are not geometrically

local, and have therefore been called “surface code with extra long-distance checks” [22]. These checks allow for a number of advantages over the surface code, such as a higher threshold for error correction and lower qubit overhead [5].

We begin by defining the following shift matrices:

$$x = P_\ell \otimes \mathbb{I}_m, \quad y = \mathbb{I}_\ell \otimes P_m,$$

where P is a cyclic permutation matrix in the form $\sum_{i=1}^\ell |i\rangle \langle i+1|$, which maps $|i+1\rangle$ to $|i\rangle$ [22]. Note that the Dirac notation here is simply used to describe the outer product and thus algebraically define these matrices, not as a reference to a specific quantum state. In addition, the matrices x and y satisfy $xy = yx$ and $x^\ell = y^m = \mathbb{I}_{\ell m}$. We use x and y to represent stepping through the grid in the x - and y -direction respectively.

Formally, the BB code is defined as a pair of $\ell m \times \ell m$ binary matrices:

$$A = A_1 + A_2 + A_3$$

$$B = B_1 + B_2 + B_3$$

where each matrix A_i, B_j is one of the shift matrices x and y taken to an integer power. For instance, we could choose:

$$A = x^3 + y + y^2$$

$$B = y^3 + x + x^2$$

These matrix polynomials are then used to define parity check matrices H^X and H^Z :

$$H^X = [A|B], \quad H^Z = [B^\top|A^\top].$$

The dimension of H^X and H^Z is thus $\ell m \times 2\ell m$. For our purposes, $2\ell m = n$, the number of physical qubits, and $\ell m = \frac{n}{2}$, which is the number of vertex and plaquette stabilizers in the grid. We use these two matrices to index which qubits in the grid are to be acted on by stabilizers. Each row contains the qubit indices for a specific stabilizer, and each column is a binary indication of whether a specific qubit in the grid is part of that stabilizer. A and B will each have exactly three non-zero entries in each row and column, such that in total, each row and column will have six nonzero entries. This means each stabilizer (row) checks six qubits, and each qubit (column) participates in six checks (three X and three Z) [5]. If we were to define the surface code in this notation, there would be four 1s per row (at most), because each stabilizer acts on only four qubits. The shifting mechanism defined by A and B is a nice, algebraic way to describe the systematic construction of an evenly-spaced pattern repeating throughout the grid, including nonlocal connections. With some appropriate indexing of the grid, we are able to specify the exact qubits to include in each stabilizer, uniquely defining a BB code.

Measuring a stabilizer corresponds to checking whether the associated parity condition set in the H^X and H^Z matrices is satisfied. Specifically, we require that

$$H^X(H^Z)^\top = 0.$$

This requirement ensures that the stabilizers commute, so that we can use a similar procedure as with the standard surface code to identify the stabilizers for which this constraint does not hold, where the anticommutation relation will let us detect stabilizers in the -1 eigenstate. The syndrome that is produced informs the corrections we make as a result.

It is common to use the notation $[[n, k, d]]$ to specify the parameters of an error correcting code, where n is the number of physical qubits, k is the number of logical qubits we encode, and d is the number of errors the code can correct. IBM, the developer of the BB code, has primarily considered a BB code it calls the *gross code*, after the term for a dozen dozen, known as a gross. Fittingly, its parameters are $[[144, 12, 12]]$: 144 physical qubits are required to encode 12 qubits, and the code can correct up to 12 errors without issue [5]. With the 1000-to-1 ratio needed for the toric surface code, the gross code (and BB codes in general) offer a clear improvement in this regard. As an additional plus, the connections between qubits in the BB code form a Tanner graph [23], which is a bipartite graph that can be separated into two disjoint sets. In this structure, nodes (qubits) within each set are only connected to members of the other, never to one another. This can be useful when physically constructing this architecture, since it allows the hardware to be split cleanly into two separate levels.

2.4 Fault-tolerant Quantum Computation

2.4.1 Universal Gate Sets

Assuming we have created some scheme for fault-tolerant storage of quantum data, we still face a number of significant challenges before we can perform any fault-tolerant computations. In particular, the *Eastin-Knill theorem* [24, 25] is commonly paraphrased as follows:

The Eastin-Knill theorem (informal statement)

No quantum error correcting code can possess a universal set of transversal logical gates.

The full proof of this theorem, given by Eastin and Knill in their paper [24], relies on group theory and the algebraic properties of the error correcting codes themselves, and is far more rigorous and technical than we need for this project. For our purposes, it is enough to understand that the theorem places restrictions on the fault-tolerant quantum computation architecture. So far, we have not encountered the terms *transversal* and *universal*, but in this section, we will define them and argue why they are useful properties for a gate and gate set to have respectively, and thus why the Eastin-Knill theorem is such an important result in quantum computing.

Before we can fully understand what the theorem is saying, though, we need to delve further into how applying gates to quantum states forms a circuit. For this, we need the concept of a *unitary matrix*, or simply a unitary, which is a norm-preserving matrix $U \in \mathbb{C}^{N \times N}$, $N \in \mathbb{N}$, whose conjugate transpose $U^\dagger$ is also its inverse, meaning $U^\dagger U = U U^\dagger = \mathbb{I}_N$. Mathematically, we can represent any quantum circuit on n qubits with a single unitary matrix $U \in \mathbb{C}^{2^n \times 2^n}$. While this representation can certainly be useful, collapsing the circuit into one matrix in this way generally does not capture a lot of important behaviors and phenomena we are interested in when studying quantum error correction. In hardware, we never apply a single giant unitary gate; instead, as we have seen in the description of Grover's algorithm in Section 2.2.1, we decompose the computation into smaller, simpler gates. In doing so, we could in principle have our pick of a large selection of possible gates, but if for simplicity we restrict ourselves to implementing only a few different types, we would like the set of gates we choose to be *universal*. A universal gate set can approximate any such unitary U to an arbitrary degree of precision with a finite sequence of gates, an important property for our gate set to have, as without it, there will always be computations we cannot perform.

Furthermore, the Eastin-Knill theorem tells us that we cannot have a universal gate set in which every gate is *transversal*. To understand what this means, we need to introduce the concept of a *code block* (or *error-correcting patch*, interchangeably), which is what we collectively call the n physical qubits encoding one or more logical qubits in a specific error-correcting code.

Transversality of a logical gate

A logical gate G_L acting on k logical qubits is said to be *transversal* if it can be implemented by applying independent physical gates between qubits at the same position across code blocks, with no interaction between qubits at different positions within a block. Formally, G_L decomposes as a tensor product over positions:

$$G_L = \bigotimes_{i=1}^n G_i^{(1,2,\dots,k)},$$

where $G_i^{(1,2,\dots,k)}$ acts only on the i -th qubit in each of the k blocks.

The transversal property is illustrated for a single- and multi-qubit gate in Fig. 2.3. As the diagram shows, transversal gates ensure single-qubit errors do not propagate to multiple qubits within an error-correcting patch, making them inherently fault-tolerant: A single error occurring when applying the gate will produce at most one error for the error-correcting patch to correct. This is clearly a desirable property for gates in our chosen gate set to have, but the Eastin-Knill theorem describes a fundamental limitation, in that any gate set we choose capable of approximating arbitrary quantum circuits will contain at least one non-transversal gate. It is important to emphasize that transversality is not an inherent property of a gate, but rather a relationship between a gate and a particular error-correcting code. Thus, we would like to choose the error-correcting architecture and gate set in concert, such that the most frequently used gate types are transversal with respect to our chosen code.

2.4.2 Choosing a Gate Set: Clifford+T

Clifford+T is a common choice of universal gate set, fittingly comprised of Clifford and *T*-gates. Clifford gates are unitaries that map Pauli operators to Pauli operators under conjugation, which unlocks some very desirable properties when performing computations in the quantum hardware (as well as when simulating classically, which will be explained in more detail in Section 3.2 in relation to our experiments).

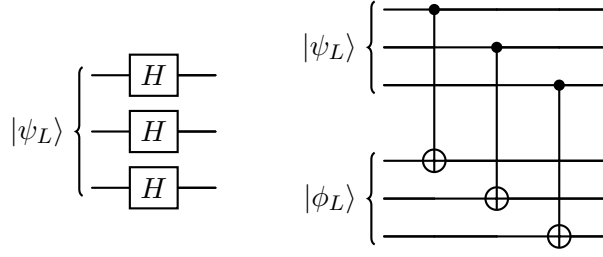


Figure 2.3: Circuit diagrams of transversal logical gates on a single-qubit gate (H) to the left and a multi-qubit gate ($CNOT$) to the right.

Recall from Section 2.3 that the stabilizers S_i are themselves tensor products of Pauli operators, and that the code space is defined by the eigenspace condition $S_i |\psi\rangle = +|\psi\rangle$. If we apply a Clifford gate C and ask whether the updated state $C|\psi\rangle$ still satisfies this condition, we find that the updated stabilizers $CS_iC^\dagger$ are again valid Pauli products simply by the definition of Clifford gates, meaning the code space is preserved. This is the algebraic reason Clifford gates are the natural choice for logical gates in stabilizer codes: They are compatible with the error-correcting architecture by construction.

In the codes we consider, the Clifford gates have been shown to admit fault-tolerant implementations [26, 27, 28]. For the surface code, the full Clifford group is strictly transversal. For the BB code, it is realized via *fold-transversal* gates, meaning the physical qubits are first permuted according to a code automorphism (a rewiring that maps stabilizers to stabilizers and logical operators to logical operators) and then single-qubit gates are applied independently to each qubit, exactly as in the transversal case. Since no individual operation ever touches more than one qubit, errors cannot propagate, and fold-transversal gates are equally fault-tolerant [27, 28].

Together, the code space preservation and transversality of Clifford gates in these codes make them well-behaved: They stay in the same algebraic space; they have inherent fault-tolerance and a conceptually simple circuit design. The T -gates, however, given by:

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix},$$

require substantially different treatment. To understand why, note that conjugation by T does not preserve the Pauli group:

$$TXT^\dagger = \frac{1}{\sqrt{2}}(X + Y),$$

which is not a Pauli operator, but rather a linear combination of Paulis. This is the fundamental incompatibility: Since stabilizers are by definition Pauli products, any operation that preserves the stabilizer formalism must map Pauli products to Pauli products under conjugation, and the T -gate violates this requirement. This also means

the T -gate is not transversal in the stabilizer codes we consider. Suppose for contradiction that T were transversal in some stabilizer code with stabilizer group $\mathcal{S}$. If so, $T^{\otimes n}$ would implement a logical gate on the code space, and would therefore be required to preserve $\mathcal{S}$ under conjugation. Both the surface code and the BB code are *Calderbank-Shor-Steane* (CSS) codes [4], meaning their stabilizer groups consist of purely X -type and purely Z -type Pauli products. Since T fixes Z exactly ($TZT^\dagger = Z$) but maps X out of the Pauli group, conjugation by $T^{\otimes n}$ maps every X -type stabilizer to an object outside the Pauli group, contradicting the requirement that $\mathcal{S}$ be preserved. Hence, T cannot be transversal in the codes we consider.

However unwieldy T -gates may be, though, they are still a requirement for universality, and form a universal gate set along with the Clifford group. They necessitate complicated workarounds to be implemented fault-tolerantly, which we return to in Section 3.3.

Chapter 3

Experiments and Analysis

3.1 General Limitations when Simulating Quantum Systems

In order to design and develop the technology necessary to begin implementing new error-correcting architectures, it is typically helpful to simulate how the components might work. In particular, a natural starting point for assessing the resource costs of a fault-tolerant implementation of Grover's algorithm is to simulate the physical qubit setup it would require. However, simulating the full fault-tolerant system with an error-correcting code of our choice turns out to be nearly impossible, even for the smallest nontrivial case of Grover's algorithm with only two input qubits.

To understand why this is so problematic, recall the expression for the unitary matrix defining an arbitrary quantum circuit with n input qubits:

$$U \in \mathbb{C}^{2^n \times 2^n}.$$

Given a gate set, we obtain U by *lifting* the individual gate matrices to the appropriate dimension $2^n \times 2^n$ and multiplying them in the right order. Specifically, in a quantum circuit, lifting a single-qubit gate $G \in \mathbb{C}^{2 \times 2}$ acting on qubit k of an n -qubit system means constructing the corresponding $2^n \times 2^n$ unitary

$$\tilde{G} = \mathbb{I}_2 \otimes \cdots \otimes G \otimes \cdots \otimes \mathbb{I}_2,$$

where G appears in the k -th tensor factor. This approach preserves the mathematical structure of the full state at all times when applying a gate to only a few of the qubits, but it comes at a cost. The exponential scaling when n grows as we add qubits to the system means that we quickly face extremely large matrices when simulating anything beyond minimal examples or the logical abstractions of algorithms. This is bad news

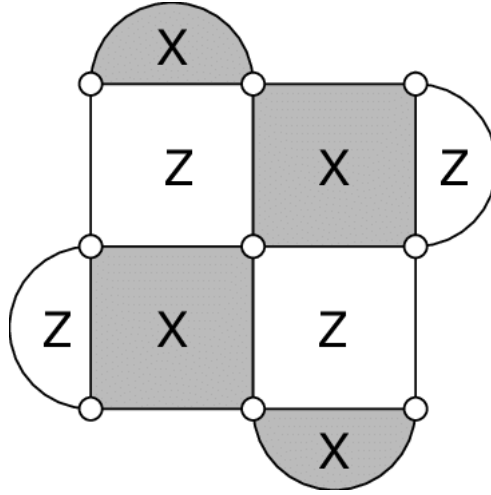


Figure 3.1: Via Fowler et al. [29]: “The smallest possible surface code, consisting of nine data qubits. The indicated eight stabilizers could, in principle, be measured sequentially by a single additional qubit, or measured in parallel by an additional eight qubits.” Note that in this visualization of the surface code, the grid is rotated 45° compared to the previous representation; the data qubits are the nine white circles instead of on the grid lines, and the ancillary check qubits belonging to each stabilizer are not shown. Each stabilizer measures the four (or two) qubits located at its corners.

for us, as adding the error-correction architectures we are primarily interested in will prevent us from being able to simulate the circuit.

For instance, the smallest possible surface code is displayed in Fig. 3.1. It requires at least ten qubits, provided we reuse a single ancillary qubit for measurement rather than using eight individual ones for each of the stabilizers. Each gate we would need in a simulation of Grover’s with two logical qubits would be a $2^{20} \times 2^{20}$ matrix to account for the no fewer than 20 qubits in the system. A single $2^{20} \times 2^{20}$ complex matrix has $2^{40} \approx 1.1 \times 10^{12}$ entries. Storing each entry as a 64-bit (8-byte) double-precision complex number (i.e. two 64-bit floats for the real and imaginary parts) therefore requires

$$2^{40} \times 16 \text{ bytes} = 2^{44} \text{ bytes} \approx 17.6 \text{ TB}$$

of memory per matrix. In practice, we rarely need to store the full unitary U explicitly; it suffices to track the state vector $|\psi\rangle \in \mathbb{C}^{2^n}$ and apply each lifted gate $\tilde{G}$ in sequence. The state vector itself is far more manageable at $2^{20} \times 16 \text{ bytes} = 16 \text{ MB}$, but each product $\tilde{G}|\psi\rangle$ requires an explicit $2^{20} \times 2^{20}$ matrix, which reintroduces the same memory bottleneck. Exploiting the tensor product structure of $\tilde{G}$ allows this to be replaced by a sequence of much smaller operations, and sparse or structured representations can reduce the constant factor, but the fundamental $\mathcal{O}(4^n)$ memory scaling of an n -qubit unitary cannot be avoided in general, and 17.6 TB far exceeds the RAM available on any single consumer machine. This estimation also assumes none of the gates necessitate additional ancilla qubits to further drive up this cost, which, as we will see in Section 3.3 when we delve into the implementation of the T -gate, is not the case.

3.1.1 Proof-of-concept Experiments with Smaller Codes

The memory requirements we have now established make a direct simulation of Grover’s algorithm on the surface code infeasible even in its smallest instance, so instead we consider alternative codes with lower qubit overhead.

Fault-tolerant physical implementations of Grover’s algorithm are only now beginning to be realized [30], and a detailed exploration of how they might look at larger scales remains an open and timely problem. The experiments in this section serve two purposes. First, they establish that the logical structure of Grover’s algorithm functions correctly under continuous error correction, and second, they demonstrate the classical simulation barrier that motivates the stabilizer-based computational methods used throughout the remainder of the thesis. Two smaller error correcting codes were selected for this purpose: the *5-qubit code* and the *7-qubit Steane code*, both of which are defined and described in more detail in Appendix A.

The 5-qubit code is the smallest quantum error correcting code capable of correcting any single-qubit error [4], making it a natural starting point. Both codes functioned as stable quantum memory under the same procedure: encoding a logical codeword, injecting a single-qubit Pauli error, measuring the syndrome and applying the corresponding correction from a lookup table. All 30 test cases for the 5-qubit code (X , Y , and Z errors on each of the five physical qubits, applied to both $|0_L\rangle$ and $|1_L\rangle$) were recovered correctly, which was confirmed by computing the *fidelity*, given by $|\langle\psi_L|\psi_{\text{corrected}}\rangle|^2$. It can be interpreted as a measure of overlap between the corrected and original states, equal to 1 when they are identical, which is what we found across all test cases. However, the 5-qubit code turns out to be a poor candidate for computation. Clifford gates are not transversal in it, meaning fault-tolerant logical operations become a challenge.

Simulating Toy Version of Grover’s Algorithm

Thus, we move on to considering the 7-qubit Steane code. Unlike in the 5-qubit code, Clifford gates are transversal in the 7-qubit code, making it a common choice in quantum error correction research today. We can verify its error correction capability in the same fashion as the 5-qubit code, this time implemented using IBM’s quantum simulator package `Qiskit` [31], rather than being coded from scratch.

Once the memory aspect worked as intended, the algorithm itself could be implemented. The logical Hadamard is $H_L = H^{\otimes 7}$, valid because the Steane code is *self-dual*: The same parity-check matrix governs both X - and Z -type stabilizers, so $H^{\otimes 7}$ exchanges them symmetrically. The logical X is the transversal $X_L = X^{\otimes 7}$, mapping $|0_L\rangle$ to $|1_L\rangle$. A logical controlled- Z between two code blocks is implemented by applying a physical CZ between each pair of corresponding qubits across the blocks.

The oracle and diffusion operators are then built entirely from these three building blocks. Notably, this gate set contains no T -gates. With only two logical qubits, the oracle reduces to a single controlled- Z , since CZ naturally applies a phase flip of -1 exactly when both qubits are $|1\rangle$, which is all a two-element oracle requires. In a cryptographically relevant implementation, the oracle would be replaced by the full AES circuit, whose AND gates decompose into Toffoli gates under the Clifford+ T gate set. This is where the T gates enter, reintroducing the overhead that dominates Jaques’ resource estimates. The experiment here is therefore a proof-of-concept for the logical structure of fault-tolerant Grover’s algorithm, with the computational core of the oracle deliberately kept simple. As such, the target state is explicitly encoded into the oracle circuit, which would not be the case in a real application. In practice, the oracle is a fixed circuit evaluating some function f , which recognizes solutions without prior knowledge of what they are.

Procedure: Grover’s algorithm on two Steane-encoded logical qubits

1. **Encode:** prepare $|0_L\rangle$ for both logical qubits using the Steane encoding circuit (14 physical qubits total)
2. **Superpose:** apply $H_L = H^{\otimes 7}$ to both blocks
3. **Oracle** (applied once, since $\lfloor \frac{\pi}{4}\sqrt{4} \rfloor = 1$):
 - (a) For each block i : if $t[i] = 0$, apply $X_L = X^{\otimes 7}$ (*flip 0-target bits to 1*)
 - (b) Apply transversal CZ_L between corresponding qubits of both blocks
 - (c) Undo the flips from step (a)
4. **Diffusion:** apply H_L, X_L, CZ_L, X_L, H_L to both blocks
5. **Measure:** compute the parity of all 7 physical qubits in each block via a single ancilla qubit

After each step of the algorithm, errors are injected, stabilizers are measured and Pauli corrections applied before the next layer is executed. The error model used is the *depolarizing noise model* [4], which applies a random Pauli error to a qubit after each gate with some fixed probability p , independent of the gate or the state of the qubit. The error probabilities $p_1 = 0.001$ for single-qubit gates and $p_2 = 0.01$ for two-qubit gates were selected to reflect the typical performance gap between one- and two-qubit operations on current superconducting hardware, where two-qubit gate fidelities are consistently an order of magnitude worse than single-qubit ones [32]. These values are representative of near-term device capabilities and place the simulation in a regime where error correction has meaningful but not overwhelming work to do: low enough that the code is operating below its threshold, but high enough that logical errors remain visible on the scale of the experiment. The idle error probability $p_{\text{idle}} = 10^{-5}$ models decoherence on qubits

3.1. General Limitations when Simulating Quantum Systems

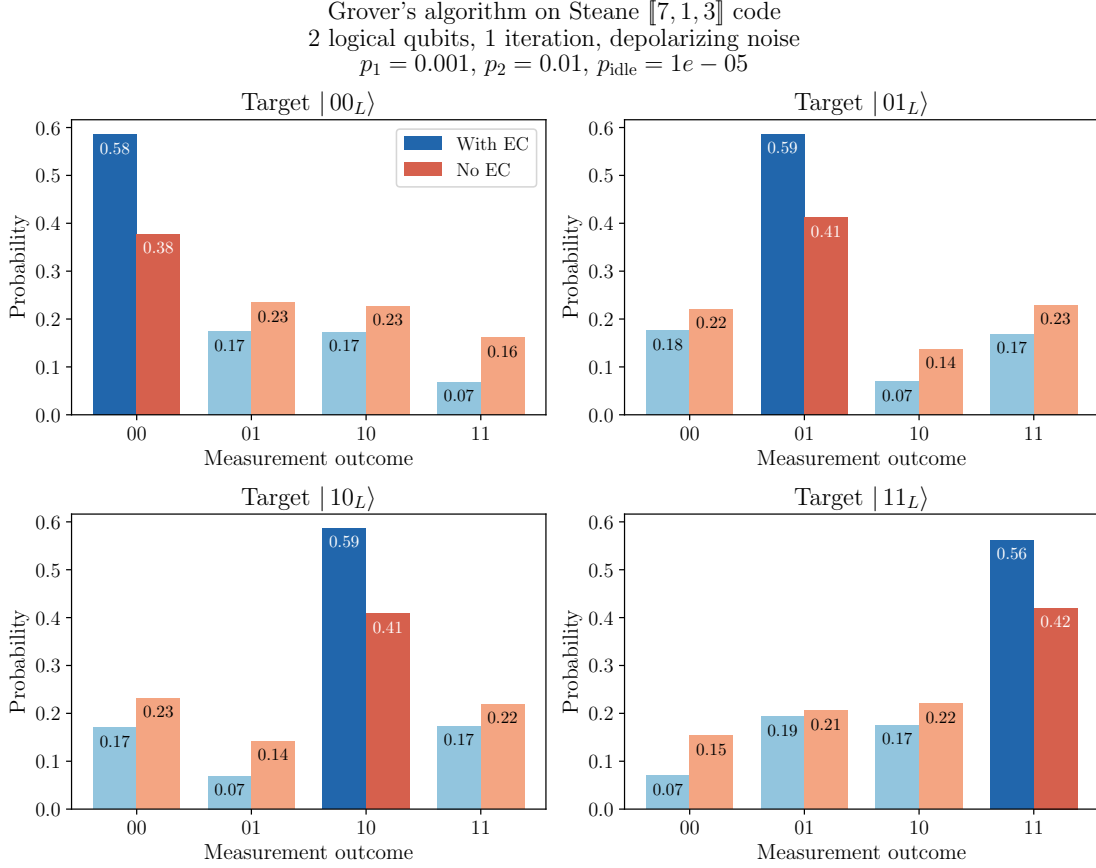


Figure 3.2: Measurement outcome distributions for Grover's algorithm on two Steane $[[7, 1, 3]]$ -encoded logical qubits (1,024 shots each) under depolarizing noise ($p_1 = 0.001, p_2 = 0.01$) and the probability of idle qubits being corrupted $p_{\text{idle}} = 10^{-5}$. Error correction is applied continuously throughout the circuit. Each panel targets one of the four logical basis states, identifying the correct state with probability ≈ 0.6 after a single Grover iteration.

that are not actively participating in a gate layer. This is set conservatively low relative to p_1 and p_2 because idle errors accumulate on a different, slower timescale than gate errors in most hardware platforms [32], and the primary purpose of this experiment is to characterize the impact of gate-level noise rather than memory-level decoherence.

Figure 3.2 shows the outcome distributions for all four target states at 1,024 shots each under this noise model. The correct state is found with probability ≈ 0.6 after a single iteration. While not a stellar result, it is still clearly an improvement over the uniform distribution that would result from a completely failed computation. The outcome distributions also match theoretical expectations in that the bitwise complement of the target (the state requiring simultaneous logical errors on both qubits) is consistently the least likely non-target outcome. Outcomes requiring only a single logical failure occur with probability scaling as p , while reaching the complement requires two independent logical failures and therefore scales as p^2 . With $p \ll 1$, the complement is suppressed by an additional factor of p relative to the single-flip outcomes.

It is worth noting that even this simplified experiment of a Clifford-only circuit with no T -gates could not be run on a general-purpose statevector simulator without exorbitant memory cost. The circuit uses 28 qubits in total: 14 data qubits across the two logical blocks, 12 error correction ancilla qubits, and two readout ancilla qubits. Simulating this classically as a full statevector requires storing 2^{28} complex amplitudes, which at double precision amounts to exactly 4 GB. This is already at the limit of what a typical consumer machine can hold for a single process, before accounting for the working memory the simulator additionally requires. The simulation is only tractable here because the `AerSimulator` in `Qiskit` exploits aspects of the stabilizer code structure introduced in Section 2.3.1 to sidestep the full vector simulation entirely. Previously, we considered stabilizers a tool for QEC, but as we will see, they turn out to be useful as a means of efficient classical simulation as well.

3.2 Utilizing Stabilizer Formalism as a Computational Tool

The exponential scaling of full unitary simulation, as described in Section 3.1, presents a fundamental barrier to classical simulation of large quantum circuits. In certain codes, however, we can at least partially circumvent the issue: For an n -qubit system, rather than storing and manipulating the full $2^n \times 2^n$ unitary, or even the state vector of 2^n complex amplitudes, a stabilizer state can be represented compactly by a set of n independent commuting Pauli operators $\{g_1, \dots, g_n\}$ called *generators*. This uniquely specifies a state $|\psi\rangle$, which is the one joint $+1$ eigenstate of all g_i simultaneously. Thus, to recover the state vector from the generators, one simply asks which state satisfies $g_i |\psi\rangle = +|\psi\rangle$ for all i . To further illustrate how the stabilizer representation lets us recover the corresponding state vector uniquely, we will step through a small example.

Example: Recovering a state vector from the stabilizer representation

Consider the 3-qubit bit-flip code, with stabilizers $g_1 = ZZI$ and $g_2 = IZZ$, and logical operators $\bar{X} = XXX$, $\bar{Z} = ZZZ$. The generators alone define the codespace, spanned by $|\bar{0}\rangle = |000\rangle$ and $|\bar{1}\rangle = |111\rangle$, but do not yet specify which logical state we are in. To define a specific state, we augment the generator list with a logical operator. If we add $\bar{Z} = ZZZ$ as a third stabilizer, the unique joint $+1$ eigenstate of all three is $|000\rangle = |\bar{0}\rangle$, since we see that $ZZZ|111\rangle = -|111\rangle$, ruling out $|111\rangle = |\bar{1}\rangle$ as it is not a $+1$ eigenstate of the set. If we add $\bar{X} = XXX$ instead, the unique joint $+1$ eigenstate becomes $\frac{1}{\sqrt{2}}(|000\rangle + |111\rangle) = |\bar{+}\rangle$. In each case, the full state vector is recoverable from the generator list alone, with no ambiguity.

The storage advantage is now clear: Each generator g_i is a tensor product of n single-qubit Paulis, requiring only $\mathcal{O}(n)$ bits to store. Since we know the number of stabilizers we need is also on the order of n , the full generator list needs only $\mathcal{O}(n^2)$ bits, compared to $\mathcal{O}(2^n)$ complex numbers for the state vector.

We also have computational shortcuts when working with Clifford gates, which we recall from Section 2.4.2 are the gates that map Pauli operators to Pauli operators under conjugation. If we have a stabilizer state $|\psi\rangle$ with generators $\{g_1, \dots, g_n\}$, we can apply a Clifford gate C to obtain a new state $C|\psi\rangle$. Rather than computing the full matrix product $C|\psi\rangle$ in a 2^n -dimensional space, we can take advantage of the stabilizer formalism and the fact that the generators uniquely determine the state to calculate the effect of C based on what happens to the generators. We want a new set of operators g'_i such that $g'_i(C|\psi\rangle) = C|\psi\rangle$, which the map $g_i \mapsto Cg_iC^\dagger$ achieves. We can verify that generators $g'_i = Cg_iC^\dagger$ defined in this way do indeed stabilize our updated state $C|\psi\rangle$, since it holds that

$$g'_i C|\psi\rangle = Cg_iC^\dagger \cdot C|\psi\rangle = Cg_i|\psi\rangle = C|\psi\rangle.$$

This stabilizer update rule holds for any unitary matrix, but working with Clifford gates unlocks the advantage of ensuring $Cg_iC^\dagger$ is always another valid Pauli string, preserving the compact $\mathcal{O}(n)$ -bit-per-generator storage throughout the entire computation. If conjugation by C could produce arbitrary matrices, we would be back to storing exponentially large objects and the efficiency gain would be lost.

To see the mechanics of the update rule, consider applying H to $|0\rangle$, stabilized by $g = Z$. We can simply update the stabilizer $HZH^\dagger = X$ and read off that the new state is stabilized by X , meaning it is the state $|+\rangle$. No matrix multiplication of a state vector is needed; the state update is captured in its entirety by relabeling one Pauli letter. In general, since a Clifford gate touches at most two qubits at a time (with the two-qubit gate $CNOT$ being the only fundamental Clifford gate acting on more than one qubit), updating one generator costs $\mathcal{O}(1)$, relabeling at most two Pauli letters according to a fixed lookup table. Updating all n generators therefore costs $\mathcal{O}(n)$ in total, compared to the $\mathcal{O}(2^n)$ cost of full state vector simulation. This is the main result of the Gottesman-Knill theorem [33]: Any circuit composed entirely of Clifford gates can be simulated efficiently on a classical computer in the stabilizer representation. This is exactly the trick the `AerSimulator` from `Qiskit` utilized to simulate our 28-qubit system in Section 3.1.1 by representing the quantum state as a set of stabilizers, rather than a full state vector.

3.2.1 Simulating Stable Quantum Memory with PanQEC

A natural place to begin our investigation into the merit of the BB code is with quantum memory. To do so, the QEC simulation and visualization package `PanQEC` [34] was used to visualize the geometry of several error correcting codes and to extract their logical

operators. It stores stabilizer codes internally in *binary symplectic format*, in which each Pauli operator on n qubits is represented as a binary vector $v = (v_X \mid v_Z) \in \mathbb{F}_2^{2n}$, the vector space of binary strings of length $2n$. Here, the first n bits indicate which qubits carry an X factor and the last n indicate which carry a Z factor. A full stabilizer group with m generators is then stored as an $m \times 2n$ binary matrix, replacing what would otherwise require tracking m dense $2^n \times 2^n$ matrices. This yields a significant reduction in memory overhead and makes operations such as checking commutativity straightforward via the symplectic inner product.

As a baseline and to verify the correctness of our PanQEC implementation, we do a quantum memory threshold simulation, shown in Fig. 3.3, where we compare the gross code to the surface code with identical logical qubit counts, in order to say something meaningful about the architectural trade-offs relevant to Jaques' assumptions. The depolarizing error model was kept constant across all experiments. In addition, all experiments use the *Belief Propagation with Localized Statistics Decoding* (BP+LSD) decoder to determine the corrections to make based on the syndrome of the error [35]. We compare logical error rate as a function of physical error rate under Monte Carlo sampling.

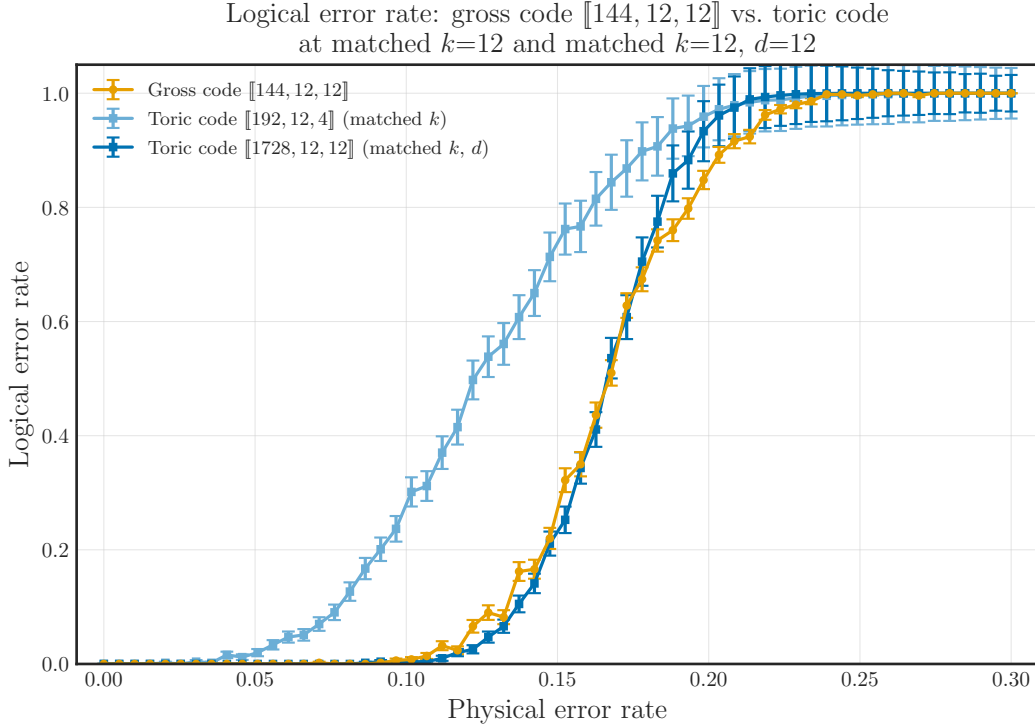


Figure 3.3: Quantum memory simulation results comparing the BB code (orange) against toric code implementations at identical logical qubit counts (light blue), as well as matching code distance (dark blue). Logical error rate is plotted as a function of physical error rate under Monte Carlo sampling.

Results

At matched logical qubit count ($k = 12$), the gross code reaches a lower logical error rate at equivalent physical error rates, consistent with its more favorable qubit overhead ratio: The surface code requires $n = 192$ physical qubits to encode 12 logical qubits at distance $d = 4$, while the gross code achieves the same k at $d = 12$ using only $n = 144$. At matched k and d , we now see that the surface code is able to correct roughly the same number of errors as the gross code, which aligns with the theory given the equal code distance. However, there is now a huge disparity in the number of physical qubits required for the surface code to do this correction, as it requires $n = 1728$ physical qubits for equivalent error protection, a factor of twelve more than the gross code. This confirms the memory-level improvement that motivates our choice of BB code as an alternative architecture, and identifies the physical error rate at which the gross code operates reliably as $p \approx 0.10$ in this experimental setup.

3.2.2 Simulating Transversal Logical Operators with PanQEC

Having established that the BB code offers an advantage over the surface code at the level of quantum memory, the natural next question is whether this advantage persists once we require the code to support logical computation. Jaques' argument, as discussed in Section 2.1, identifies the cost of fault-tolerant gate implementation (specifically, the T-gate) as the dominant resource bottleneck, not memory overhead. The memory result therefore motivates but does not resolve the question of whether BB codes meaningfully improve the outlook for Grover's algorithm in practice. To begin probing this, we return to PanQEC, which has functionality for determining logical operators of various types, identifying which physical qubits we would have to modify to correspond to a gate application on the logical qubits. We call this set of qubits the *gate support* of the logical gate, which has a *weight* equal to the number of qubits in the set w .

The experiment proceeds as follows. For both the BB code and the toric surface code, both instantiated at $n = 98$ physical qubits ($L_x = L_y = 7$), the PanQEC logical operator matrices $\mathbf{l}_X$ and $\mathbf{l}_Z$ are extracted directly from the code object. Each row of $\mathbf{l}_X$ is a binary vector of length n identifying the physical qubits on which a representative $\bar{X}$ operator acts. The minimum-weight row is selected as the gate support, which we interpret as the most qubit-efficient realization of the logical gate.

To simulate gate noise, X -errors are injected independently on each gate-support qubit with probability p , modeling the scenario where each physical gate operation fails with that probability. No errors are placed on the remaining $n - w$ qubits, reflecting the principle that each gate application is an independent opportunity for error [4]. Since errors are injected only on the support qubits, we isolate the effect of gate noise so any

logical failure is directly attributable to the gate, not to background memory errors. This is a narrower question than the full fault-tolerance threshold of Bravyi et al. [5], which models circuit-level noise across every operation in the syndrome measurement cycle, but it is the relevant one here: The BB code’s heavier gate support means errors are concentrated on more qubits per gate application; it is not obvious a priori that its higher distance compensates for this. The BP-LSD decoder determines corrections, and the residual error is checked against $\mathbf{I}_Z$ to assert logical failure. This will only occur when the decoder is fooled, which happens when the error pattern, combined with whatever correction the decoder applies, leaves a residual that is a logical operator $\bar{X}$. Since $\bar{X}$ commutes with all stabilizers by definition, it produces no syndrome and passes through the decoder undetected, silently flipping the logical state.

10^6 Monte Carlo trials are run per error rate. A distance- d code corrects any error pattern of weight up to $\lfloor \frac{d-1}{2} \rfloor$, so the smallest uncorrectable pattern has weight $d_e = \lfloor \frac{d-1}{2} \rfloor$, which we call the error dimension [26]. Since each independent error occurs with probability p , the leading-order logical failure probability scales as $p_L \propto p^{d_e}$ in the low- p region. With 10^6 trials per point, the smallest resolvable failure probability is on the order of 10^{-6} , where only a few failures are observed. The lowest measured points therefore carry large relative uncertainty and several fall at or near this resolution floor. The power-law fit is used to extrapolate below the floor into the low- p region where direct Monte Carlo cannot resolve individual logical failures.

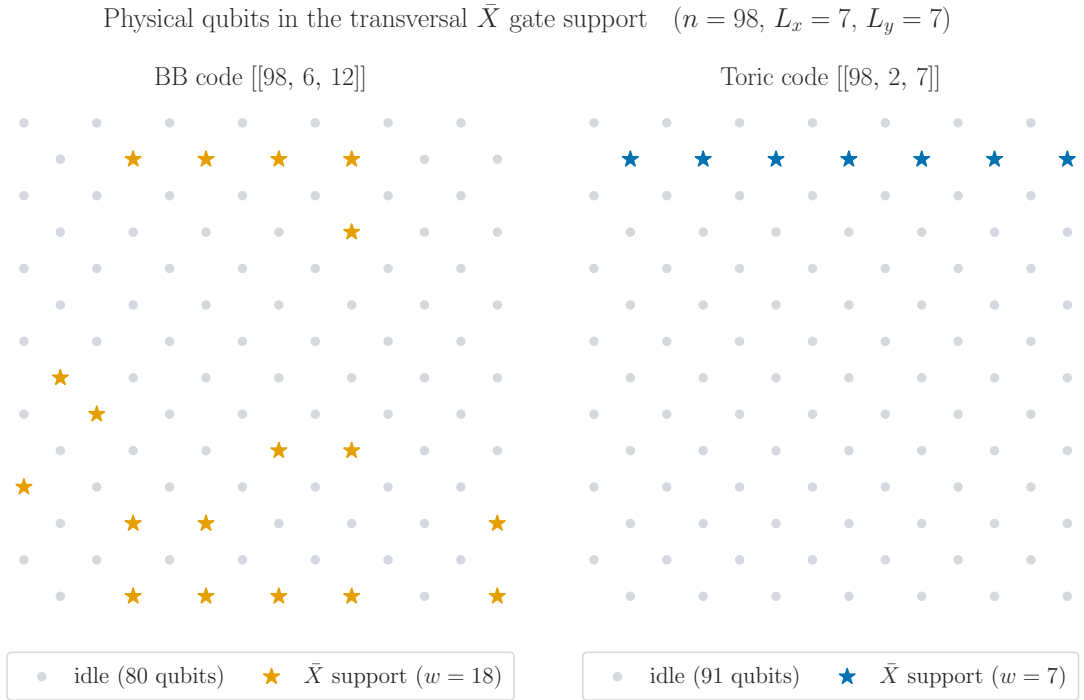


Figure 3.4: Physical qubits activated during a transversal $\bar{X}$ gate (colored stars) versus idle qubits (gray), for the BB code $[[98, 6, 12]]$ (left, yellow) and the toric surface code $[[98, 2, 7]]$ (right, blue), both at $n = 98$ physical qubits.

Results

Figure 3.4 displays the gate support of the minimum-weight $\bar{X}$ representative the decoder identified on the physical lattice for each code. For the toric surface code $[[98, 2, 7]]$, the gate support weight equals the code distance ($w = d = 7$): The gate support is exactly the minimum-weight logical X representative, so the shortest path implementing the logical gate is also the easiest path for the decoder to confuse with a logical error. For the $[[98, 6, 12]]$ BB code [27], the decoder returns $\bar{X}$ representatives of weight 18, which is the gate support used in the simulation. These are not guaranteed to be the minimum-weight representatives of their logical class, which is an NP-hard problem in general, but they are valid logical operators and represent a concrete gate implementation.

Figure 3.5 shows the logical error rate of the codes after injecting errors on the individual qubits in the gate support, followed by the decoder's corrections. Results are plotted on a logarithmic scale to resolve the small failure probabilities at realistic error rates. Through repeated trials across a wide range of physical error rates, it was determined that a crossover occurs near $p \approx 0.30$, above which the surface code becomes more tolerant. This crossover is a consequence of two competing effects: A larger gate support creates more potential error sites (disadvantaging the BB code), while higher code distance allows more errors to be corrected (favoring it). However, since current superconducting

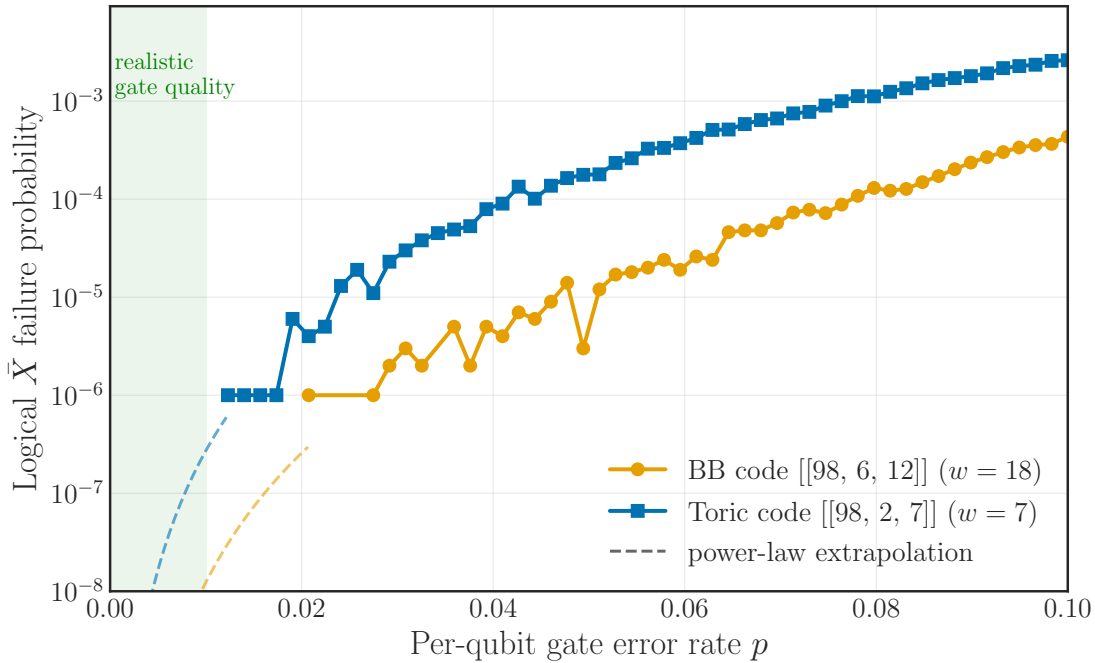


Figure 3.5: Logical $\bar{X}$ failure probability (log scale) as a function of per-qubit gate error rate p , for the BB code $[[98, 6, 12]]$ (yellow, $w = 18$) and toric surface code $[[98, 2, 7]]$ (blue, $w = 7$), both at $n = 98$ physical qubits. Dashed lines show a power-law fit $p_L \propto p^{d_e}$ to the observed data, extrapolated into the low- p regime where 10^6 Monte Carlo trials are insufficient to resolve individual logical failures. The exponent is the error dimension $d_e = \lfloor \frac{d-1}{2} \rfloor$ [26].

two-qubit gate error rates are typically below 1% [36, 1], the physically relevant regime lies well to the left of this crossover, and Fig. 3.5 is therefore plotted only up to $p = 0.10$ to focus on this region.

We see that throughout this regime, the BB code $[[98, 6, 12]]$ achieves lower logical failure probability than the toric code $[[98, 2, 7]]$, despite its gate support being about 2.6 times larger ($w = 18$ versus $w = 7$). The code's higher distance ($d = 12$ versus $d = 7$) more than compensates for the larger number of gate-error sites. Furthermore, provided the operators we identified are in fact the minimum-weight operators, the heavier gate support of the BB code also means that the probability of enough simultaneous errors to cause a logical failure scales as a higher power of p , making undetected logical failure exponentially less likely at realistic error rates.

3.3 The Trouble with the T -gate

As established in Section 2.4.2, the T -gate's incompatibility with the Pauli group under conjugation means it cannot be implemented by any manipulation of the stabilizer structure alone. This has a direct consequence for classical simulation: The moment a T -gate is introduced into a circuit, the stabilizer update rule $g_i \mapsto Cg_iC^\dagger$ no longer produces a valid Pauli string, the compact $\mathcal{O}(n^2)$ stabilizer representation breaks down, and we must fall back to full state vector simulation at exponential cost. This is why the efficient classical simulation guaranteed by the Gottesman-Knill theorem for Clifford circuits, discussed in Section 3.2, does not extend to circuits containing T -gates.

The standard route to a fault-tolerant T -gate is *magic state distillation* (MSD) [37]. The central idea is to separate the non-Clifford content of a computation from its execution: rather than applying a T -gate directly to a data qubit at runtime, one instead prepares a state $|T\rangle = T|+\rangle$ in advance using only noisy physical operations, which we call a *magic state*. Then, through a method known as *gate teleportation*, the magic state is consumed, and we effectively transfer it from the ancilla qubit to a data qubit by entangling the two and then measuring the ancilla [4]. In short, this is a way to apply a non-Clifford (non-transversal) gate to a data qubit without direct application, at the cost of measuring (and thus destroying) the state prepared in the ancilla. All the necessary non-Clifford action is concentrated into preparing the magic state in advance; the rest of the computation then proceeds using only Clifford gates and measurements.

The difficulty is that magic states prepared directly from noisy physical operations are themselves noisy, and consuming a low-fidelity magic state introduces errors into the computation. MSD resolves this by taking multiple noisy copies of $|T\rangle$ and combining them through a carefully designed Clifford circuit to produce a smaller number of copies with substantially higher fidelity. The dominant protocol for distillation of $|T\rangle$ is the

15-to-1 scheme [37]. In it, 15 noisy input magic states are consumed to produce a single output state whose error rate scales as the cube of the input error rate, providing cubic suppression at the cost of a fifteen-fold resource overhead. The physical unit that executes this protocol is called a *magic state factory*, and in large-scale fault-tolerant architectures, it is common for the majority of physical qubits to be devoted to running factories rather than to the logical computation itself [8]. This distillation can also be applied iteratively. Letting p_{input} denote the error rate of the noisy input magic states, feeding the output of one round as input to the next yields error rates scaling as $p_{\text{input}}^{3^L}$ after L levels, at a resource cost of 15^L input states per output [37].

Recent work by Xu et al. [8] designs concrete MSD factories tailored to the BB architecture, providing direct resource comparisons against surface code factories. At a physical error rate of $p_{\text{phys}} = 10^{-3}$, their gross-code-based factories require substantially fewer physical qubits than a surface code factory achieving the same target output error rate. Crucially, however, the total space-time volume of magic state production (the product of physical qubits and syndrome extraction cycles, and a more complete measure of total resource cost) is comparable between the two architectures. The BB code does not reduce the time required to produce a single magic state, but rather it achieves the same output quality at a smaller physical footprint, with the cycle cost remaining broadly similar. For a fixed hardware budget, this smaller footprint in principle allows more factories to run in parallel, potentially improving T -gate throughput, but each factory still takes a comparable number of syndrome cycles per magic state produced. They further identify BB code factories as particularly well-suited as the second stage in a two-level distillation pipeline, following an initial step performed on a surface code.

3.4 Assessing the Quantum Threat

In Section 2.1, we reviewed Jaques’ argument against the quantum threat to AES, which rests on three distinct pillars: First, the high physical-to-logical qubit ratio obscures the true scale of engineering required to construct working quantum computers that can run Grover’s algorithm. Second, fault-tolerant gates carry a significant computational cost. Third, Grover’s algorithm parallelizes badly. Together, these factors make the circuit conceptually more complex, and increase both its resource requirements and total runtime. The specifics of Grover’s poor parallelism (which is unrelated to the choice of error-correcting code) were covered in Section 2.2, and the experiments in Sections 3.2 and 3.3 were designed to probe the other pillars of the argument under the assumption of the BB code instead of the surface code. However, in this section, we will argue that even with the improvements afforded by the BB code, constructing stable physical qubits is not the true limitation, as Jaques also points out.

The memory threshold results of Section 3.2.1 show that the BB code is capable of reducing the physical qubit overhead substantially: At matched logical qubit count and code distance, the surface code requires twelve times as many physical qubits as the gross code. The gate noise results of Section 3.2.2 indicate that this advantage persists under gate-level error injection, but more marginally. Therefore, adopting the BB code arguably brings the necessary physical qubit requirements closer to a conceivable scale. The physical-to-logical qubit ratio is not necessarily fixed at 1000:1; under the gross code, it is closer to 12:1 at equivalent error protection.

The gate count for implementing Grover’s algorithm on AES has been an active area of optimization since the original estimates of Grassl et al. [38], who designed a circuit requiring on the order of 2^{86} Clifford gates and 2^{86} T -gates in total across all Grover iterations. The only place where T -gates occur is in the oracle, specifically in the S-box step of AES [39]. The gate counts are more or less architecture-agnostic, besides the fact that constructing a circuit based on the Clifford+ T gate set in the first place is considered a suitable choice for surface codes. Subsequent work has been able to reduce this figure: Jaques et al. [39] introduced depth-optimized oracle circuits and showed that fewer AES evaluations are required per oracle call, reducing the total T -gate count. More recent circuit optimizations [12, 13] pushed the improvements further, placing the current best estimate for the total serial T -gate count required for the full Grover circuit at approximately 2^{80} .

Unlike classical gates, quantum gates are processes rather than physical devices [1]. This raises the important question of how much time it will take to execute this circuit in full. To make a lower bound estimate, we begin by simplifying our calculation with the assumption that operations corresponding to Clifford gates are essentially free, meaning we can focus on the 2^{80} figure for T -gate cost. Another assumption we make is that individual quantum operations will be at least as slow as current classical operations. By NIST’s estimation, current¹ classical computing architectures can perform 2^{64} operations serially in a decade [40]. Using their estimate, the naive calculation of the time it will take to perform 2^{80} operations is thus:

$$2^{80} = 2^{16+64} = 2^{16} \cdot 2^{64} = 65,536 \cdot 2^{64}.$$

With 2^{64} operations serially per decade, it will take 655,360 years to complete the calculation of the T -gates alone. However, this number assumes no parallelism whatsoever, and the 2^{80} T -gate figure assumes a single machine searching the full 2^{128} key space. One might hope, then, that partitioning the key space among P machines reduces the per-machine circuit depth, which turns out to be true: Each machine is assigned a partition of $\frac{2^{128}}{P}$ keys, which becomes its effective search space $N' = \frac{2^{128}}{P}$.

¹as of 2017

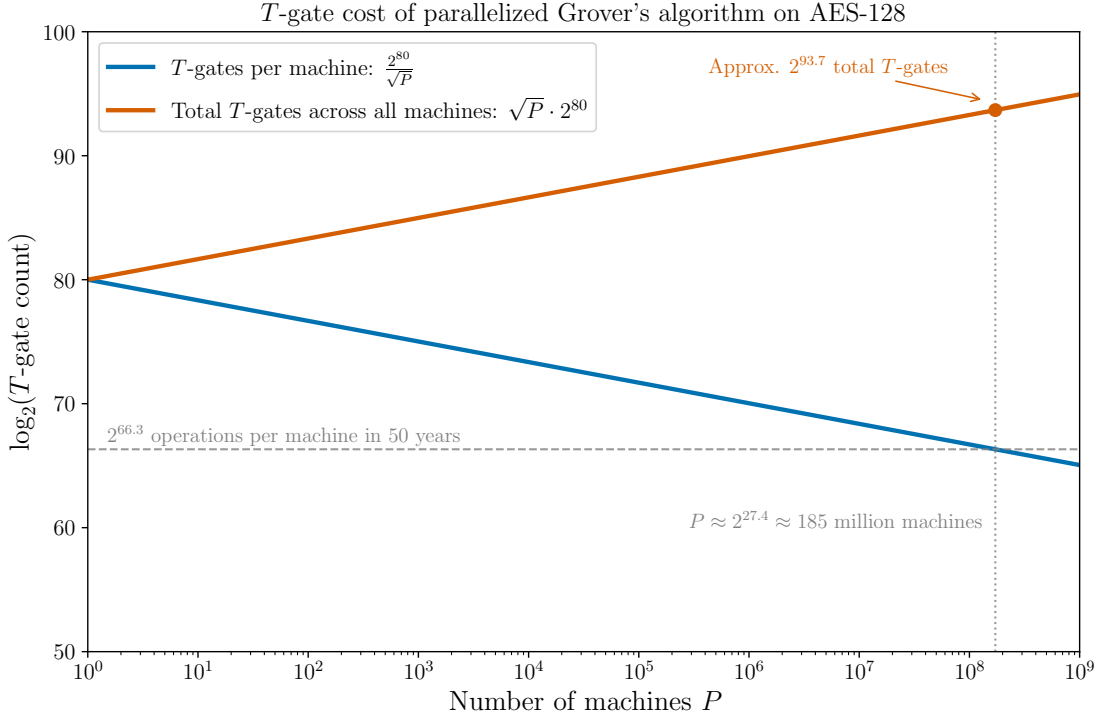


Figure 3.6: T -gate cost of Grover's algorithm on AES-128 as a function of the number of machines P , each searching an equal partition of the 2^{128} key space. The per-machine cost (blue) decreases as $2^{80}/\sqrt{P}$, while the total cost across all machines (orange) increases as $\sqrt{P} \cdot 2^{80}$. The horizontal line marks the 50-year viability threshold of $2^{66.3}$ T -gates per machine, which would require approximately $2^{27.4} \approx 185$ million machines (vertical line). At this point, the total T -gate count across all machines reaches $2^{93.7}$, nearly 2^{14} times the naive single-machine estimate of 2^{64} .

The number of Grover iterations t' required to search this reduced space is determined by

$$t' = \left\lfloor \frac{\pi}{4\theta'} \right\rfloor, \quad \theta' = \sin^{-1} \left(\sqrt{\frac{1}{N'}} \right) \approx \frac{\sqrt{P}}{2^{64}},$$

giving $t' \approx \frac{2^{64}}{\sqrt{P}}$, a reduction by a factor of $\sqrt{P}$ relative to the single-machine case. Since the T -gate cost of each oracle call is fixed, the per-machine T -gate count reduces proportionally to $\frac{2^{80}}{\sqrt{P}}$. However, as established in Section 2.2.2, this relief comes at a cost. The total T -gate count across all machines is $P \cdot \frac{2^{80}}{\sqrt{P}} = \sqrt{P} \cdot 2^{80}$, which grows with P . Figure 3.6 shows both quantities as functions of P .

As a concrete example, suppose we define a viable attack as one in which each machine completes its share of the computation in under 50 years, i.e. at most $5 \times 2^{64} \approx 2^{66.3}$ T -gate operations per machine. Setting $\frac{2^{80}}{\sqrt{P}} = 2^{66.3}$ and solving for P gives $P = 2^{27.4} \approx 185$ million machines. At this point, the total T -gate count across all machines is $\sqrt{P} \cdot 2^{80} = 2^{93.7}$, which is nearly ten billion times the naive quantum estimate of 2^{64} . Parallelization does not resolve the runtime problem; it can at best translate a time limitation to a question of hardware, but at an inordinately large scale, while substantially worsening the runtime problem in aggregate.

The impact of the BB code on these figures, as established in Section 3.3, is to reduce the physical qubit cost of a 15-to-1 magic state factory by roughly a factor of 12 [8]. However, it does so by increasing the number of syndrome cycles per factory run, leaving the space-time volume of magic state production comparable to the surface code equivalent. The qubit saving translates in principle to more parallel factories under a fixed hardware budget, but this does not reduce the number of T -gates the circuit requires, nor the time each factory takes to produce a single magic state. Thus, there is no evidence of a reduction in the resources required to construct the machinery we would need at this enormous scale. A constant multiplicative improvement to the physical qubit cost does not change the exponential scaling of the total resource requirement.

A natural question is whether code switching could circumvent this limitation. In fact, the 15-to-1 MSD scheme already exploits exactly this idea by using 15-qubit Reed-Muller code, in which the T -gate is transversal, in order to create magic states [37]. This is not a cheap operation. The factory is resource-intensive, and as established in Section 3.3, the majority of physical qubits in a large-scale fault-tolerant architecture are devoted to running these factories rather than to the logical computation itself. More fundamentally, no architectural choice of this kind changes the serial T -gate count of 2^{80} , which is determined entirely by the structure of the AES oracle and the Clifford+ T circuit decomposition it requires.

To truly emphasize the infeasibility of this setup, recall also that this lower bound assumes the 2^{86} Clifford gates in this circuit design are completely negligible, and that the *hundreds of millions* of quantum machines we require access to all run at the speed of current classical computers. Each of these is already a significant overestimate of near-term capability, and they do not all hold at once. As of 2025, the largest quantum processors contain on the order of a few thousand physical qubits [41], placing the required scale many orders of magnitude beyond the current state of the art. It is possible that in building the machines we would need to break AES with Grover’s algorithm, using the BB code architecture over the surface code architecture could grant a $12\times$ improvement in terms of physical qubits, but there is nothing about using the BB code specifically that addresses the runtime limitation, seeing as it does not reduce the number of T -gates we must apply serially. The prognosis for Grover’s algorithm for AES is therefore technically better with something like the gross code—the 185 million machines we would need could possibly be built with 12 times fewer qubits than if we were to use the surface code—but clearly, the engineering gap remains enormous in absolute terms.

This forms the central conclusion of this thesis: Even with the adoption of BB codes, either the computation runs for an astronomically long time, or reducing the runtime to a conceivable scale will have physical architecture requirements that the choice of the error-correcting code cannot address, since the code determines qubit cost per machine, not how many machines we need. The only way to make any improvement here is to

address the estimate of 2^{80} T -gates applied serially, and whether an error-correcting code with a different native transversal gate set admits a more efficient implementation of the AES oracle. If it can be expressed more efficiently, the 2^{80} figure could in principle be reduced not by a constant factor but by a change in the dominant term, which is the only kind of improvement that would materially alter the conclusion reached here. BB codes will not be a good candidate for this use case, since they retain the same Clifford+ T logical gate structure as the surface code, with non-transversal T -gates handled identically through magic state distillation.

Chapter 4

Conclusion

This thesis set out to test whether Jaques’ argument against the quantum threat to AES-128 is specific to the surface code, or whether it survives under a more favorable error-correcting architecture. To probe this, we examined the Bivariate Bicycle code family as an alternative, running experiments and analysis at three levels: quantum memory, transversal gate noise, and magic state distillation.

Before turning to the BB code experiments, the proof-of-concept experiments of Section 3.1.1 established two important baselines. First, they confirmed that direct statevector simulation of even the smallest fault-tolerant instance of Grover’s algorithm is classically intractable at realistic qubit counts, motivating the stabilizer-based simulation methods used throughout. Second, a working implementation of Grover’s algorithm on two Steane-encoded logical qubits under depolarizing noise demonstrated that the logical structure of the algorithm functions correctly under continuous error correction, identifying the target state with probability ≈ 0.6 after a single iteration—a clear improvement over the uniform distribution that would result from a failed computation, and consistent with theoretical expectations.

We began the exploration of the BB code as an alternative to surface codes by utilizing PanQEC to simulate quantum memory. Here, the BB code offers a clear improvement: At matched logical qubit count and code distance, the surface code requires twelve times as many physical qubits as the BB code for equivalent error protection, confirming the resource advantage that motivates the BB code as an alternative architecture. This advantage persists at the level of gate noise, where the BB code achieves lower logical failure rates than the surface code throughout the practically achievable error rate regime, despite its heavier gate support, because its higher code distance more than compensates. At the level of MSD, Xu et al. [8] show that BB code-based factories achieve comparable space-time volume to surface code factories at a smaller physical qubit footprint, and are particularly well-suited as the second stage in a two-level distillation pipeline.

BB codes therefore reduce the overhead at every level we can measure. And yet, the fundamental bottleneck identified by Jaques remains entirely untouched. The T -gate cost of running Grover’s algorithm on AES-128 is set by the number of serial T -gate operations the circuit requires, currently estimated at around 2^{80} , and by how long each magic state factory takes to produce a single magic state. The BB code reduces the physical qubit cost of each factory by roughly a factor of twelve, but leaves the cycle cost of magic state production and the total T -gate count unchanged. Under the parallelization analysis of Section 3.4, meeting even a 50-year runtime threshold requires on the order of 185 million machines, at a total T -gate cost of $2^{93.7}$ across all machines. A constant-factor improvement to the per-factory qubit count does not alter this picture.

Jaques’ conclusion therefore survives under the assumption of a more resource-efficient architecture. His argument rests on three independent pillars: the physical-to-logical qubit ratio, the cost of fault-tolerant gate implementation, and the poor parallelism of Grover’s algorithm. The BB code somewhat weakens the first, and marginally weakens the second via qubit savings in magic state factories. The third is a property of the algorithm itself and is untouched by any architectural choice. Importantly though, none of the three pillars are truly broken. The significance of this result is not that the BB code fails to help, but that it allows us to identify with more confidence where improvement would actually matter: not in the choice of error-correcting code, but in the serial T -gate count itself. Closing the gap would require either substantially better MSD protocols, or an error-correcting code whose native transversal gate set admits a more efficient implementation of the AES oracle than Clifford+ T does, reducing the dominant term in the resource estimate, not merely its constant factor.

4.1 Future Work

Several natural extensions of this work remain unexplored. First, the memory threshold experiment of Section 3.2.1 utilized a simple depolarizing noise model. Extending the comparison between the BB code and the surface code to more realistic noise models (including correlated errors, measurement noise, and qubit reset errors [42, 43]) would give a more accurate picture of the architectural trade-offs in practice. In particular, measurement and qubit reset are among the noisiest operations in real superconducting hardware [42], and omitting them causes the error correction results to appear more favorable than they would be on physical devices.

Second, the gate noise experiment of Section 3.2.2 used a fixed system size of $n = 98$ physical qubits. A natural follow-up is a code distance scaling study: How does the logical failure rate of the BB code improve as qubits are added, compared to the surface code? This would clarify whether the advantage observed here is robust as the codes scale, and could be compared against existing theoretical predictions.

Third, all experiments in this thesis used the BP+LSD decoder. The BB code advantage observed in Section 3.2.2 depends on the decoder’s ability to correctly identify minimum-weight error patterns on a non-geometrically-local code, which is a harder decoding problem than for the surface code. Whether this advantage is robust across different decoders, and what efficiency and accuracy trade-offs different decoding strategies introduce for BB codes specifically, is an open question with direct relevance to practical implementation.

Bibliography

- [1] Samuel Jaques. *Quantum Attacks on AES*. Presentation at the 26th International Conference on Cryptographic Hardware and Embedded Systems (CHES 2024). Available online: https://ches.iacr.org/2024/Jaques_CHES_2024.pdf. July 2024.
- [2] Wikipedia contributors. *Symmetric-key algorithm* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Symmetric-key_algorithm&oldid=1341644974. [Online; accessed 1-June-2026]. 2026.
- [3] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. en. May 2023. DOI: 10.6028/NIST.FIPS.197-upd1. URL: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=936594.
- [4] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.
- [5] Sergey Bravyi et al. “High-threshold and low-overhead fault-tolerant quantum memory.” In: *Nature* 627.8005 (Mar. 2024), pp. 778–782. ISSN: 1476-4687. DOI: 10.1038/s41586-024-07107-7. URL: <http://dx.doi.org/10.1038/s41586-024-07107-7>.
- [6] Wikipedia contributors. *Stabilizer code* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 28-January-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Stabilizer_code&oldid=1332747113.
- [7] Arthur Pesah. *The stabilizer trilogy I — Stabilizer codes*. Blog post. Jan. 31, 2023. URL: <https://arthurpesah.me/blog/2023-01-31-stabilizer-formalism-1/> (visited on 01/28/2025).
- [8] Shifan Xu et al. *Distilling Magic States in the Bicycle Architecture*. 2026. arXiv: 2602.20546 [quant-ph]. URL: <https://arxiv.org/abs/2602.20546>.
- [9] Emma Storberg. *TEK5550 Mandatory Assignment 1*. Unpublished midterm exam. University of Oslo, 2025.
- [10] John Watrous. *Grover’s Algorithm*. Online course. IBM Quantum Learning. 2023. URL: <https://quantum.cloud.ibm.com/learning/en/courses/fundamentals-of-quantum-algorithms/grover-algorithm/analysis> (visited on 06/01/2026).
- [11] Huinan Chen et al. *Quantum circuit for implementing AES S-box with low costs*. 2025. arXiv: 2503.06097 [quant-ph]. URL: <https://arxiv.org/abs/2503.06097>.

- [12] Liao-Liang Jiang et al. “Constructing resource-efficient quantum circuits for AES.” In: *Frontiers in Physics* 13 (2025). ISSN: 2296-424X. DOI: 10.3389/fphy.2025.1582819. URL: <https://www.frontiersin.org/journals/physics/articles/10.3389/fphy.2025.1582819>.
- [13] Qun Liu et al. *Improved Quantum Circuits for AES: Reducing the Depth and the Number of Qubits*. Cryptology ePrint Archive, Paper 2023/1417. 2023. URL: <https://eprint.iacr.org/2023/1417>.
- [14] Charles H. Bennett et al. “Strengths and Weaknesses of Quantum Computing.” In: *SIAM Journal on Computing* 26.5 (Oct. 1997), pp. 1510–1523. ISSN: 1095-7111. DOI: 10.1137/s0097539796300933. URL: <http://dx.doi.org/10.1137/S0097539796300933>.
- [15] Christof Zalka. “Grover’s quantum searching algorithm is optimal.” In: *Physical Review A* 60.4 (Oct. 1999), pp. 2746–2751. ISSN: 1094-1622. DOI: 10.1103/physreva.60.2746. URL: <http://dx.doi.org/10.1103/PhysRevA.60.2746>.
- [16] Wikipedia contributors. *Quantum decoherence — Wikipedia, The Free Encyclopedia*. [Online; accessed 20-April-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Quantum_decoherence&oldid=1345559227.
- [17] Google Quantum AI and Collaborators. “Quantum error correction below the surface code threshold.” In: *Nature* 638.8052 (Feb. 2025), pp. 920–926. DOI: 10.1038/s41586-024-08449-y. URL: <https://www.nature.com/articles/s41586-024-08449-y>.
- [18] IBM Quantum. *Other code families*. Accessed: 2025-01-28. IBM Quantum Learning. 2025. URL: <https://quantum.cloud.ibm.com/learning/en/courses/foundations-of-quantum-error-correction/quantum-code-constructions/other-code-families>.
- [19] A. Yu. Kitaev. “Quantum computations: algorithms and error correction.” In: *Russian Mathematical Surveys* 52.6 (1997), pp. 1191–1249. DOI: 10.1070/RM1997v052n06ABEH002155.
- [20] Wikipedia contributors. *Pauli matrices — Wikipedia, The Free Encyclopedia*. [Online; accessed 28-January-2026]. 2025. URL: https://en.wikipedia.org/w/index.php?title=Pauli_matrices&oldid=1327001669.
- [21] Craig Gidney et al. “Yoked surface codes.” In: *Nature Communications* 16.1 (May 2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-59714-1. URL: <http://dx.doi.org/10.1038/s41467-025-59714-1>.
- [22] Anton Brekke and Brage A. Trefjord. *Sommerprosjekt 2025*. Summer student project report on GitHub. [Online; accessed 28-Jan-2026]. 2025. URL: <https://github.com/Bragit123/QEC/blob/main/rapport.pdf>.
- [23] Wikipedia contributors. *Tanner graph — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Tanner_graph&oldid=1296985959. [Online; accessed 28-January-2026]. 2025.

- [24] Bryan Eastin and Emanuel Knill. “Restrictions on Transversal Encoded Quantum Gate Sets.” In: *Physical Review Letters* 102.11 (Mar. 2009). ISSN: 1079-7114. DOI: 10.1103/physrevlett.102.110502. URL: <http://dx.doi.org/10.1103/PhysRevLett.102.110502>.
- [25] Wikipedia contributors. *Eastin–Knill theorem* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 26-January-2026]. 2025. URL: https://en.wikipedia.org/w/index.php?title=Eastin%E2%80%93Knill_theorem&oldid=1314440925.
- [26] Austin G. Fowler et al. “Surface codes: Towards practical large-scale quantum computation.” In: *Physical Review A* 86.3 (Sept. 2012). ISSN: 1094-1622. DOI: 10.1103/physreva.86.032324. URL: <http://dx.doi.org/10.1103/PhysRevA.86.032324>.
- [27] Jens Niklas Eberhardt and Vincent Steffan. *Logical Operators and Fold-Transversal Gates of Bivariate Bicycle Codes*. 2024. arXiv: 2407.03973 [quant-ph]. URL: <https://arxiv.org/abs/2407.03973>.
- [28] Hasan Sayginel et al. *Fault-Tolerant Logical Clifford Gates from Code Automorphisms*. 2025. arXiv: 2409.18175 [quant-ph]. URL: <https://arxiv.org/abs/2409.18175>.
- [29] Austin G. Fowler et al. *Scalable extraction of error models from the output of error detection circuits*. 2014. arXiv: 1405.1454 [quant-ph]. URL: <https://arxiv.org/abs/1405.1454>.
- [30] Friederike Butt et al. “Demonstration of measurement-free universal logical quantum computation.” In: *Nature Communications* 17 (2026), p. 995. DOI: 10.1038/s41467-026-68533-x.
- [31] Ali Javadi-Abhari et al. *Quantum computing with Qiskit*. 2024. DOI: 10.48550/arXiv.2405.08810. arXiv: 2405.08810 [quant-ph].
- [32] Zhiyuan Li et al. “Error per single-qubit gate below 10^{-4} in a superconducting qubit.” In: *npj Quantum Information* 9.1 (Nov. 2023). ISSN: 2056-6387. DOI: 10.1038/s41534-023-00781-x. URL: <http://dx.doi.org/10.1038/s41534-023-00781-x>.
- [33] Daniel Gottesman. *The Heisenberg Representation of Quantum Computers*. 1998. arXiv: quant-ph/9807006 [quant-ph]. URL: <https://arxiv.org/abs/quant-ph/9807006>.
- [34] panqec. *panqec: Simulation and visualization of quantum error correcting codes*. Version main. 2021. URL: <https://github.com/panqec/panqec> (visited on 02/18/2026).
- [35] Timo Hillmann et al. “Localized statistics decoding for quantum low-density parity-check codes.” In: *Nature Communications* 16.1 (Sept. 2025). ISSN: 2041-1723. DOI: 10.1038/s41467-025-63214-7. URL: <http://dx.doi.org/10.1038/s41467-025-63214-7>.
- [36] Yao-Yao Jiang et al. “Advancements in superconducting quantum computing.” In: *National Science Review* 12 (June 2025). DOI: 10.1093/nsr/nwaf246.

- [37] Sergey Bravyi and Alexei Kitaev. “Universal quantum computation with ideal Clifford gates and noisy ancillas.” In: *Physical Review A* 71.2 (Feb. 2005). ISSN: 1094-1622. DOI: 10.1103/physreva.71.022316. URL: <http://dx.doi.org/10.1103/PhysRevA.71.022316>.
- [38] Markus Grassl et al. *Applying Grover’s algorithm to AES: quantum resource estimates*. 2015. arXiv: 1512.04965 [quant-ph]. URL: <https://arxiv.org/abs/1512.04965>.
- [39] Samuel Jaques et al. *Implementing Grover oracles for quantum key search on AES and LowMC*. Cryptology ePrint Archive, Paper 2019/1146. 2019. DOI: 10.1007/978-3-030-45724-2_10. URL: <https://eprint.iacr.org/2019/1146>.
- [40] National Institute of Standards and Technology. *Submission Requirements and Evaluation Criteria for the Post-Quantum Cryptography Standardization Process*. Tech. rep. National Institute of Standards and Technology, Dec. 2016. URL: <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf>.
- [41] Hannah J. Manetsch et al. “A tweezer array with 6,100 highly coherent atomic qubits.” In: *Nature* 647.8088 (Sept. 2025), pp. 60–67. ISSN: 1476-4687. DOI: 10.1038/s41586-025-09641-4. URL: <http://dx.doi.org/10.1038/s41586-025-09641-4>.
- [42] Sascha Heußen, David F. Locher, and Markus Müller. “Measurement-Free Fault-Tolerant Quantum Error Correction in Near-Term Devices.” In: *PRX Quantum* 5.1 (Feb. 2024). ISSN: 2691-3399. DOI: 10.1103/prxquantum.5.010333. URL: <http://dx.doi.org/10.1103/PRXQuantum.5.010333>.
- [43] György P. Gehér et al. *To reset, or not to reset – that is the question*. 2025. arXiv: 2408.00758 [quant-ph]. URL: <https://arxiv.org/abs/2408.00758>.
- [44] Andrew M. Steane. “Multiple-particle interference and quantum error correction.” In: *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 452.1954 (Nov. 1996), pp. 2551–2577. ISSN: 1471-2946. DOI: 10.1098/rspa.1996.0136. URL: <http://dx.doi.org/10.1098/rspa.1996.0136>.
- [45] Victor V. Albert, Philippe Faist, et al. “[[7, 1, 3]] Steane code.” In: *The Error Correction Zoo*. Ed. by Victor V. Albert and Philippe Faist. 2024. URL: <https://errorcorrectionzoo.org/c/steane>.

Appendix A

Definitions of Smaller Error Correcting Codes Used

In addition to the aforementioned architectures that are viable options in the landscape of quantum technology today, this project also necessitated some smaller quantum codes to allow for easier simulation of circuitry.

A.1 The 5-Qubit Code

The 5-qubit code [4] is the smallest quantum error correcting code capable of correcting an arbitrary single-qubit error. Its parameters are $[[5, 1, 3]]$, meaning five physical qubits are used to encode a single logical qubit, with a code distance of three, allowing it to detect and correct any single-qubit error.

That five qubits is the minimum required for this purpose is not immediately obvious, but follows from a counting argument. Correcting an arbitrary single-qubit error on n qubits requires distinguishing between $3n + 1$ error syndromes (the three error types X , Y and Z on each of the n qubits, plus the no-error case). With $n - k$ stabilizer generators, the syndrome space has 2^{n-k} elements. For $k = 1$ logical qubit, we therefore require $2^{n-1} \geq 3n + 1$, which is first satisfied at $n = 5$.

The four stabilizer generators of the 5-qubit code are:

$$g_1 = XZZXI$$

$$g_2 = IXZZX$$

$$g_3 = XIXZZ$$

$$g_4 = ZXIXZ$$

Appendix A. Definitions of Smaller Error Correcting Codes Used

These generators, together with their products, define the codespace. The logical operators are $\bar{X} = XXXXX$ and $\bar{Z} = ZZZZZ$.

While the 5-qubit code is attractive for its minimality, it has a significant practical limitation: The Clifford gates are not transversal in this code. This means that implementing even computationally simple logical operations requires coupling physical qubits within the code block, removing the fault-tolerance guarantees that transversality would otherwise provide. For this reason, the 5-qubit code is well-suited as a proof-of-concept for quantum memory, but not as a platform for fault-tolerant computation, which motivated the move to the 7-qubit Steane code when performing experiments.

A.2 The 7-Qubit Steane Code

The 7-qubit Steane code [44, 45] is a $[[7, 1, 3]]$ code, encoding one logical qubit in seven physical qubits with code distance three, and is one of the most well-studied codes in quantum error correction. Like the surface code and the BB codes introduced in Section 2.3, it belongs to the CSS family [4], meaning its stabilizer generators separate cleanly into an X -type set and a Z -type set.

The six stabilizer generators are:

$$\begin{aligned} g_1 &= IIIXXX \\ g_2 &= IXXIII \\ g_3 &= XIXIXI \\ g_4 &= IIIZZZ \\ g_5 &= IZZIII \\ g_6 &= ZIZIZI \end{aligned}$$

where g_1, g_2, g_3 are the X -type generators and g_4, g_5, g_6 are the Z -type generators. The logical operators are $\bar{X} = XXXXXX$ and $\bar{Z} = ZZZZZZ$.

The T -gate, as discussed in Section 3.3, is not transversal in the Steane code, as is guaranteed by the Eastin-Knill theorem, and its implementation requires the magic state distillation techniques described there.